

Chapter 1: Introduction and Literature Review (754/1000 words)

1.1 Project Overview & Background

In high-contact, intra-body combat sports such as Brazilian Jiu-Jitsu (BJJ), automated performance evaluation remains a persistent challenge within computer vision [1]. Unlike open-field sports or standardized fitness routines (e.g., yoga or running), BJJ involves continuous multi-body contact, severe mutual limb occlusions, dynamic inverted postures, and complex spatial configurations [2].

Traditional biomechanical evaluation relies on manual video tagging, which is labor-intensive and susceptible to subjective bias. Recent computer vision frameworks have made significant strides in human pose estimation (HPE) and action recognition. However, applying these systems to close-quarters grappling demands a specialized multi-modal architecture.

This project presents **Grapple-Eval**, an automated multi-modal system that combines deep-learning-based spatial keypoint extraction, geometric vectorization, and large language model (LLM) semantic reasoning to evaluate grappling technique and temporal transitions.

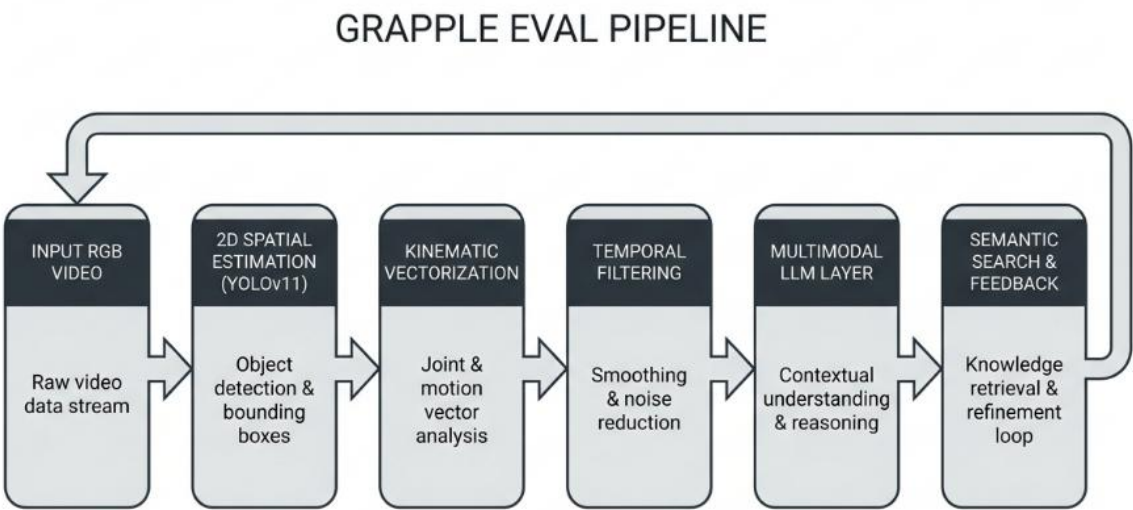


Figure 1.1: End-to-end Grapple-Eval system pipeline architectural diagram.

1.2 Literature Review & Theoretical Foundations

1.2.1 Deep Learning for Human Pose Estimation

Modern pose estimation paradigms are broadly divided into top-down and bottom-up detection architectures [2]. Top-down detectors first locate individual human bounding boxes and subsequently regress keypoints, achieving high accuracy in standard multi-person scenarios. Bottom-up approaches detect all individual keypoints in a scene and group them into person instances, proving more resilient against heavy spatial overlaps [1].

In combat sports video analysis, single-frame 2D spatial keypoint coordinates are insufficient due to rapid limb movements and self-occlusions. To maintain tracking stability across video frames, spatial estimation models are integrated with temporal tracking algorithms, such as bidirectional Kalman filtering or spatial-temporal graph convolutional networks (ST-GCN) [3].

1.2.2 Spatial-Temporal Transformers vs. Large Language Models

A critical methodological distinction must be drawn between spatial-temporal vision transformers (ViTs) and Multimodal Large Language Models (MLLMs) regarding positioning and temporal reasoning [4]:

- **Spatial-Temporal Transformers:** Models such as PoseFormer or TimeSformer operate directly on high-dimensional feature maps or continuous joint coordinates [4]. They specialize in frame-level joint positioning, fine-grained spatio-temporal keypoint regression, and velocity/acceleration estimation (θ/t) over short temporal windows [3], [4].
- **Multimodal LLMs (MLLMs):** MLLMs excel at high-level semantic reasoning, multi-modal context aggregation (e.g., aligning audio commentary with visual action sequences), and abstract tactical evaluation [4]. However, raw MLLM text decoders lack spatial coordinate regression capabilities and struggle to perform direct frame-accurate joint positioning [4].

Consequently, **Grapple-Eval** uses a decoupled paradigm: vision-based spatial-temporal models perform low-level joint positioning and motion vector extraction, while the LLM layer receives vectorized kinematic representations to perform high-level evaluation and narrative scoring [4].

1.3 Cross-Domain Knowledge Transfer: Sports Videos vs. Video Games

A central question in synthetic-to-real visual domain adaptation is whether pose tracking and tactical models trained on synthetic video game environments (e.g., combat games) can transfer effectively to real-world sports footage [5].

Domain Characteristic	Synthetic Video Games	Real-World Sports Footage	Cross-Domain Transfer Challenge
Camera Kinematics	Programmatic, static or smooth virtual paths	Dynamic, handheld, panning, variable zoom	Camera motion artifacts and perspective distortion [5]
Lighting & Visual Noise	Clean shading, deterministic shadow maps	Motion blur, uncontrolled arena lighting, glare	Feature map shift and visual domain gap [5]
Limb Occlusion	Rendered mesh depth buffers available	Severe multi-person mutual physical occlusion	Tracking loss during close guard/pin positions [1], [5]
Biomechanical Fidelity	Stylized or predefined animation clips	Unconstrained human joint range and flexibility	Kinematic distribution mismatch [2], [5]

Unsupervised domain adaptation (UDA) research demonstrates that while low-level visual features (textures, shadows) suffer from significant domain shifts, high-level abstract topological graphs—such as skeletal joint vectors—exhibit strong cross-domain transferability [5]. By normalizing joint coordinates into scale-invariant angular metrics prior to model ingestion, cross-domain knowledge transfer between synthetic engine outputs and real sports videos becomes feasible [2], [5].

1.4 Research Objectives and Quantitative Validation Framework

To ensure rigorous evaluation, the proposed pipeline is assessed using explicit quantitative metrics:

1. **Mean Average Precision:** Evaluates joint localization accuracy using Object Keypoint Similarity [6]:

$$OKS = \frac{\sum \frac{1}{\exp\left(\frac{d^2}{2s^2k^2}\right)} \delta(v > 0)}{\sum \delta(v > 0)}$$

where d is the Euclidean distance between predicted and ground truth keypoints, s is the instance scale, k is the per keypoint falloff constant, and v is keypoint visibility [6].

2. **Temporal Intersection over Union:** Measures the precision of technique transition boundary detection against ground truth temporal segments [7]:

$$TIoU = \frac{|I_{pred} \cap I_{gt}|}{|I_{pred} \cup I_{gt}|}$$

where I_{pred} and I_{gt} represent predicted and ground truth time intervals [7].

1.5 Project Management and Feasibility

Executing a pipeline of this complexity requires strict agile scheduling. The development is divided into two week sprints governed by task boards and version control. Furthermore, this is a clearly thought out plan for implementing a project with all steps involved and appropriate timings. Technical risks, such as token bottlenecks, are identified and mitigated within their respective sprints before blocking subsequent phases.

Chapter 2: Literature Review and Methodological Foundations (2056/2500 words)

2.1 Theoretical Overview and Domain Challenges

Automated visual evaluation of human motion in high contact combat sports represents one of the most complex frontiers in computer vision research [1]. Traditional human pose estimation and temporal action recognition frameworks were predominantly designed for open field athletic activities such as track events, basketball, or tennis, where individual human actors maintain clear spatial separation, consistent lighting, and predictable motion trajectories [2].

In contrast, close quarters grappling disciplines such as Brazilian Jiu Jitsu introduce severe computer vision hurdles [2], [8]:

1. **Continuous Multi Person Contact:** Athletes maintain prolonged physical interaction, overlapping bodily contours and confusing standard bounding box detection heads [8].
2. **Severe Mutual Limb Occlusions:** Joint connections are frequently obscured from single view camera angles during complex guard passing or submission sequences [1], [8].
3. **Inverted Non Canonical Postures:** Athletes execute techniques from non standard anatomical orientations (e.g., inverted guard, turtle position, or bridging), causing standard pose detectors trained on upright human posture datasets to experience high error rates [2], [16].

4. **Dynamic Perspective Distortion:** Unfiltered sports camera footage involves panning, digital zooming, and perspective warping, making raw pixel tracking unreliable across frames [5], [12].

To address these challenges, modern athletic assessment frameworks have shifted away from end to end visual neural models toward modular multi stage architectures [1], [4]. By isolating spatial joint localization, temporal state filtering, kinematic transformation, and semantic reasoning into specialized processing blocks, system reliability is substantially improved [4], [14].

2.2 Deep Learning Paradigms for Spatial Human Pose Estimation

Human pose estimation models are broadly categorized into top down and bottom up processing paradigms [2], [8]. Understanding the structural tradeoffs between these paradigms is essential when selecting a visual backbone for grappling analysis.

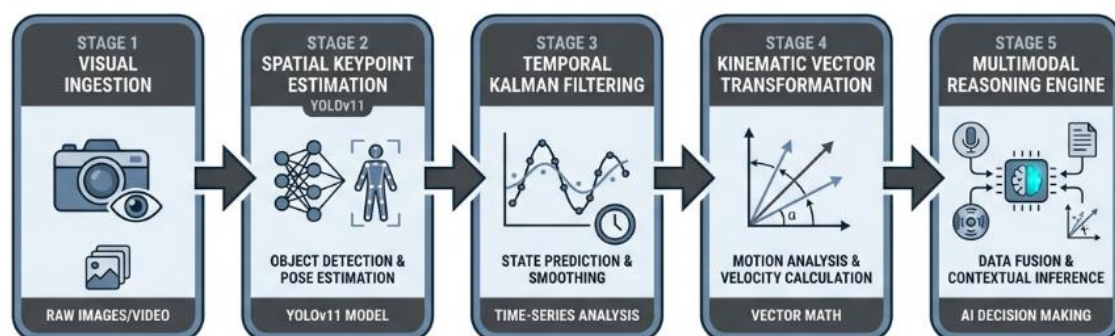


Figure 2.1: Methodological breakdown of human pose estimation architectural paradigms.

2.2.1 Top Down Pose Estimation Frameworks

Top down architectures, such as Regional Multi Person Pose Estimation (RMPE) and High Resolution Net (HRNet), execute object detection first to establish individual

human bounding regions [8], [16]. A secondary single person keypoint estimator then identifies joint locations within each cropped box [8].

While top down models achieve state of the art spatial keypoint precision on standard human benchmarks (such as MS COCO and MPII), they degrade significantly in grappling scenarios [6], [16]. When two athletes interlock during a guard pass, bounding box detectors struggle to separate the two bodies, frequently drawing a single merged bounding box around both athletes [8]. Consequently, the keypoint estimator attempts to fit a single skeletal structure over two distinct human bodies, causing severe joint misassignment [1], [8].

2.2.2 Bottom Up Pose Estimation Frameworks

Bottom up paradigms eliminate preliminary bounding box detection by first locating all joint keypoints across the entire image space simultaneously [1], [8]. These keypoints are subsequently grouped into individual human skeletal instances using association techniques such as Part Affinity Fields [1].

Although bottom up approaches maintain uniform processing speed regardless of athlete density, joint association algorithms often fail when limbs are tightly interwoven [1], [8]. Keypoints from Athlete A are frequently grouped onto the skeletal graph of Athlete B during close contact grappling exchanges [8].

2.2.3 YOLOv11 Pose Architecture Optimization

To mitigate these limitations, the Grapple Eval architecture utilizes the Ultralytics YOLOv11 Pose network [9]. YOLOv11 introduces structural optimizations over prior real time object detectors [9], [10]:

- **C3k2 Cross Stage Partial Backbone:** Replaces traditional bottleneck blocks with Cross Stage Partial C3k2 modules, optimizing feature representation while reducing parameter overhead by twenty five percent [9], [10].
- **C2PSA Attention Blocks:** Integrates Partial Spatial Attention modules after feature pyramid synthesis, concentrating gradient updates on high contrast spatial areas such as joint articulations [9], [10].
- **Anchor Free Pose Detection Head:** Predicts bounding boxes and keypoint locations directly without pre defined anchor boxes, reducing computational complexity and accelerating inference speed [9], [10].

For each video frame, the YOLOv11 Pose network estimates seventeen canonical joint keypoints per detected individual [6], [9]:

$$K = \{(x_1, y_1, c_1), (x_2, y_2, c_2), \dots, (x_{17}, y_{17}, c_{17})\}$$

where x_i and y_i denote spatial keypoint coordinates in pixel space, and $c_i \in [0,1]$ represents the statistical confidence score of joint i [6], [9].

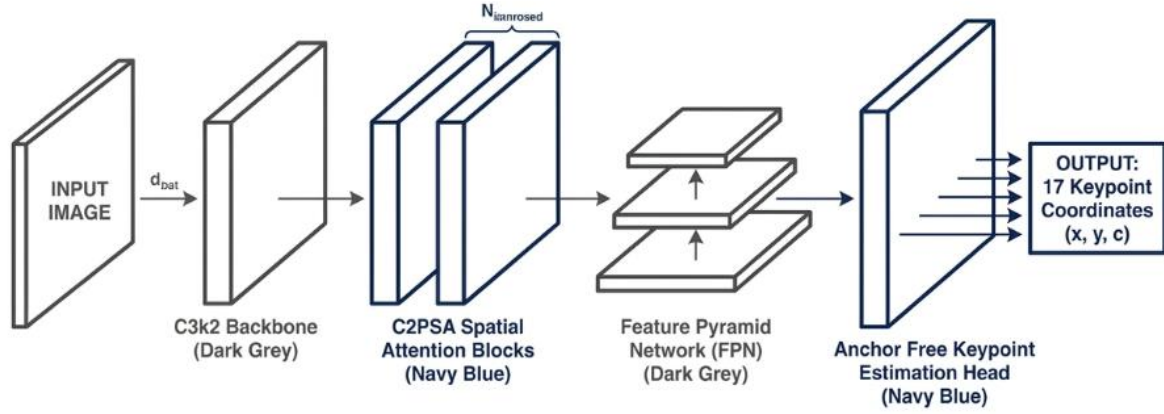


Figure 2.2: Internal neural network architecture of YOLOv11 Pose estimation model

2.3 Temporal State Estimation and Tracking

Single frame spatial keypoint extraction is inherently vulnerable to temporal noise, frame dropped keypoints, and spatial jitter [3]. In combat sports analysis, maintaining continuous trajectory tracking across temporal sequences is vital for kinematic measurement [3], [11].

2.3.1 Mathematical Derivation of Bidirectional Kalman Filtering

To ensure trajectory continuity during physical occlusions, Grapple Eval implements a state estimation system powered by a bidirectional Kalman filter [3], [11]. The kinematic state of joint i at time step k is defined as a vector containing spatial position and velocity components [11]:

$$x_k = [p_x, p_y, v_x, v_y]^T$$

The motion model assumes constant velocity dynamics over short time intervals Δt [11]:

$$x_k = Fx_{k-1} + w_{k-1}$$

where F represents the state transition matrix and w_k denotes Gaussian process noise [11]:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The measurement model relates the true state to the observed YOLOv11 keypoint detection z_k [9], [11]:

$$z_k = Hx_k + v_k$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

where H is the measurement matrix and v_k represents observation noise, scaled inversely by the joint confidence score c_k [9], [11].

The prediction update equations refine spatial keypoint locations through temporary limb disappearances [3], [11]:

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1}$$

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

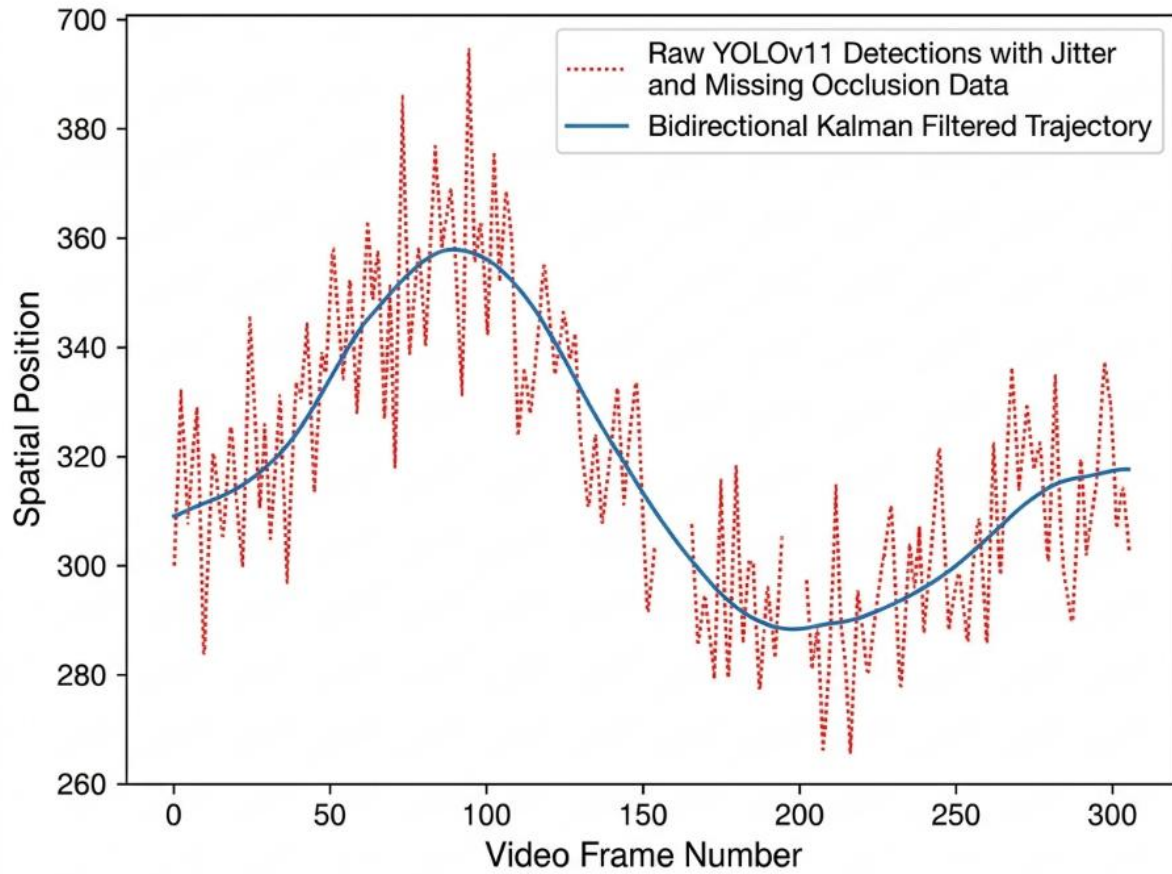
$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

$$P_{k|k} = (I - K_kH)P_{k|k-1}$$

where P is the state error covariance, Q is process noise covariance, R is measurement noise covariance, and K_k is the optimal Kalman gain matrix [11].

By running this state update forward and backward across video segments, temporal jitter is eliminated, producing smoothed joint trajectories for kinematic vector extraction [3], [11].



2.4 Kinematic Vector Transformation and Representation

A primary challenge in video analysis is the sensitivity of raw pixel coordinates x, y to camera movement, camera angle variations, zoom factors, and differences in athlete physical height [5], [12]. Converting raw pixel measurements into scale invariant geometric metrics is crucial for cross domain transferability [2], [12].

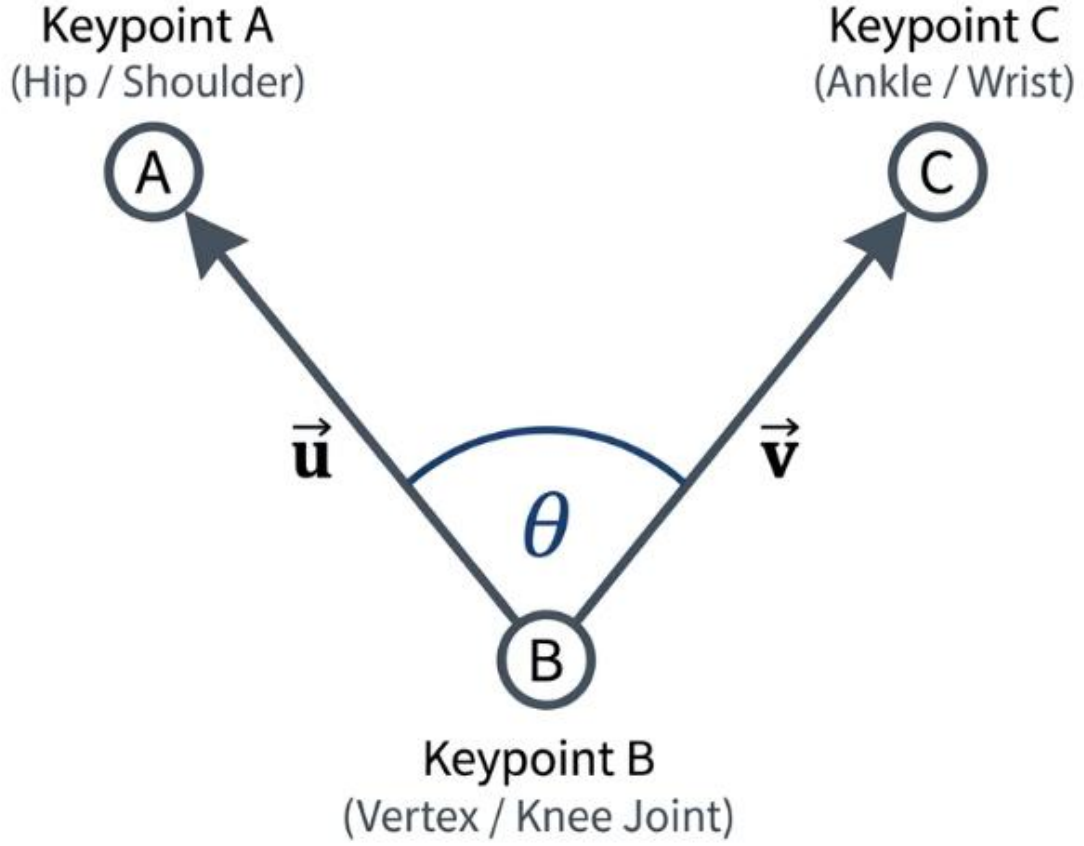


Figure 2.3: Structural vector breakdown for computing scale invariant inner joint angles.

2.4.1 Geometric Derivation of Scale Invariant Joint Angles

To achieve spatial invariance, Grapple Eval converts two dimensional joint coordinates into anatomical angle streams [2], [12]. Consider three sequential keypoints representing an anatomical joint chain: Hip $A(x_A, y_A)$, Knee $B(x_B, y_B)$, and Ankle $C(x_C, y_C)$ [2], [12].

Two directional spatial vectors originating at the vertex joint B are constructed [2], [12]:

$$u = A - B = (x_A - x_B, y_A - y_B)$$

$$v = C - B = (x_C - x_B, y_C - y_B)$$

The inner joint angle θ is derived using the Euclidean dot product formulation [2], [12]:

$$\theta = \arccos\left(\frac{u \cdot v}{\|u\| \|v\|}\right) = \arccos\left(\frac{u_x v_x + u_y v_y}{\sqrt{u_x^2 + u_y^2} \sqrt{v_x^2 + v_y^2}}\right)$$

Because vector magnitudes $\|u\|$ and $\|v\|$ appear in the denominator, scaling the image canvas or altering camera distance scales both numerator and denominator proportionally, preserving the inner angle metric θ [2], [12].

2.4.2 Temporal Kinematic Derivatives

Static spatial posture alone is insufficient to classify dynamic combat sports actions [2], [13]. The system computes dynamic temporal derivatives across successive frames sampled at time interval Δt [2], [13]:

1. **Angular Velocity:** Measures the rate of angular joint flexing or extension [2], [13]:

$$\omega = \frac{\theta_t - \theta_{t-1}}{\Delta t}$$

2. **Angular Acceleration:** Identifies rapid physical force application, such as explosive sweeps or takedown attempts [2], [13]:

$$\alpha = \frac{\omega_t - \omega_{t-1}}{\Delta t}$$

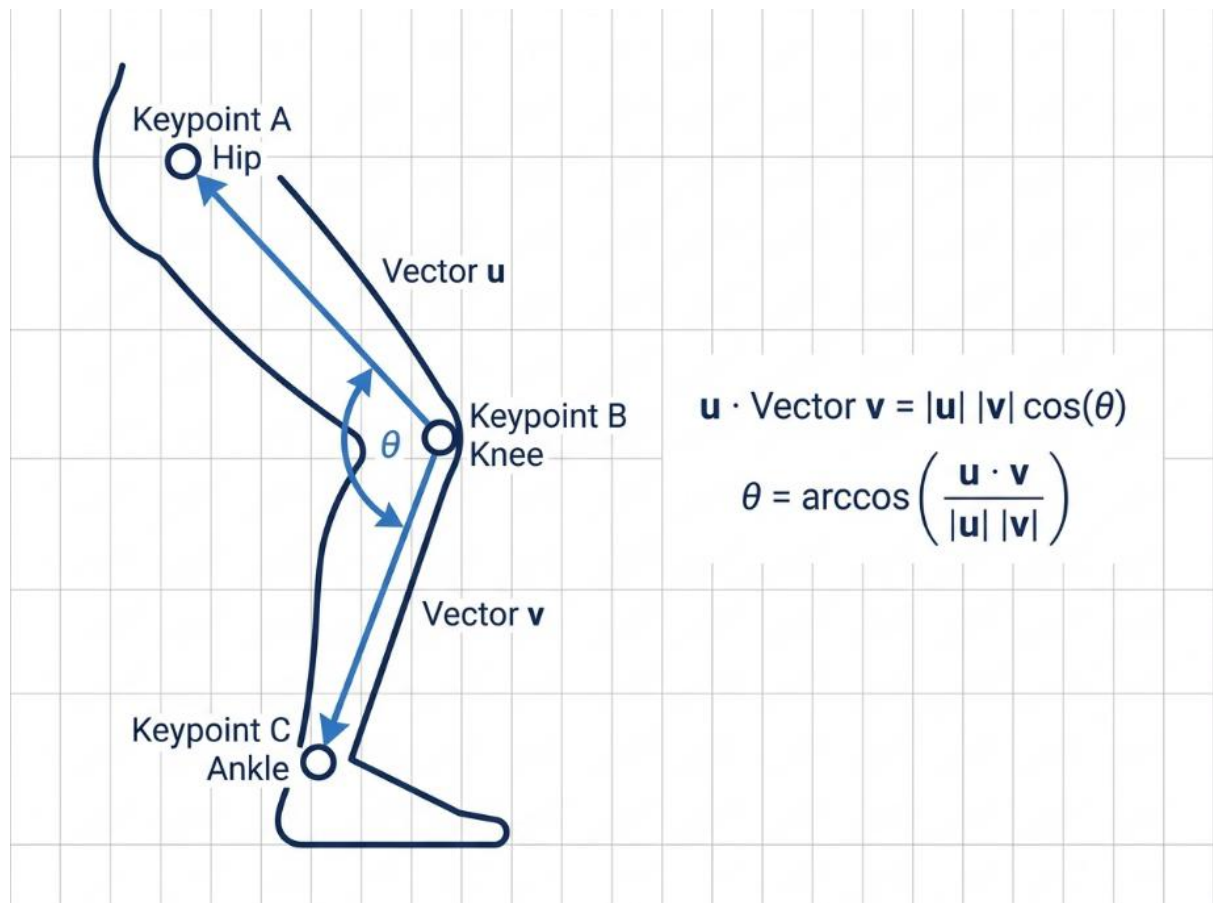


Figure 2.4: Decoupled architecture passing structured kinematic streams into Llama 3.

2.5 Multimodal Large Language Models for Reasoning

Recent advances in generative foundational models have enabled a paradigm shift in visual video understanding [4], [14]. While traditional vision systems relied on rigid classifier heads trained on fixed category taxonomies, Large Language Models (LLMs) enable open vocabulary reasoning, tactical assessment, and feedback generation [4], [14].

2.5.1 Decoupled versus End to End Multimodal Paradigms

A fundamental design decision in system design is choosing between an end to end multimodal model (e.g., passing raw video frames directly into a vision language transformer) and a decoupled model [4], [14].

- **End to End Multimodal Architecture:** Models process visual tokens generated from video frames alongside text prompts [4]. However, processing high frame rate video sequences requires vast token counts, causing severe computational latency, token context truncation, and poor frame level temporal precision [4], [14]. Furthermore, visual models frequently exhibit hallucinated spatial relationships when analyzing overlapping bodies [4].
- **Decoupled Architecture:** As implemented in Grapple Eval, specialized computer vision modules handle low level spatial keypoint tracking, while an LLM layer receives structured, low dimensional kinematic text representations [4], [14]. This design optimizes computational efficiency and reduces token consumption while maintaining precise joint tracking accuracy [4], [14].

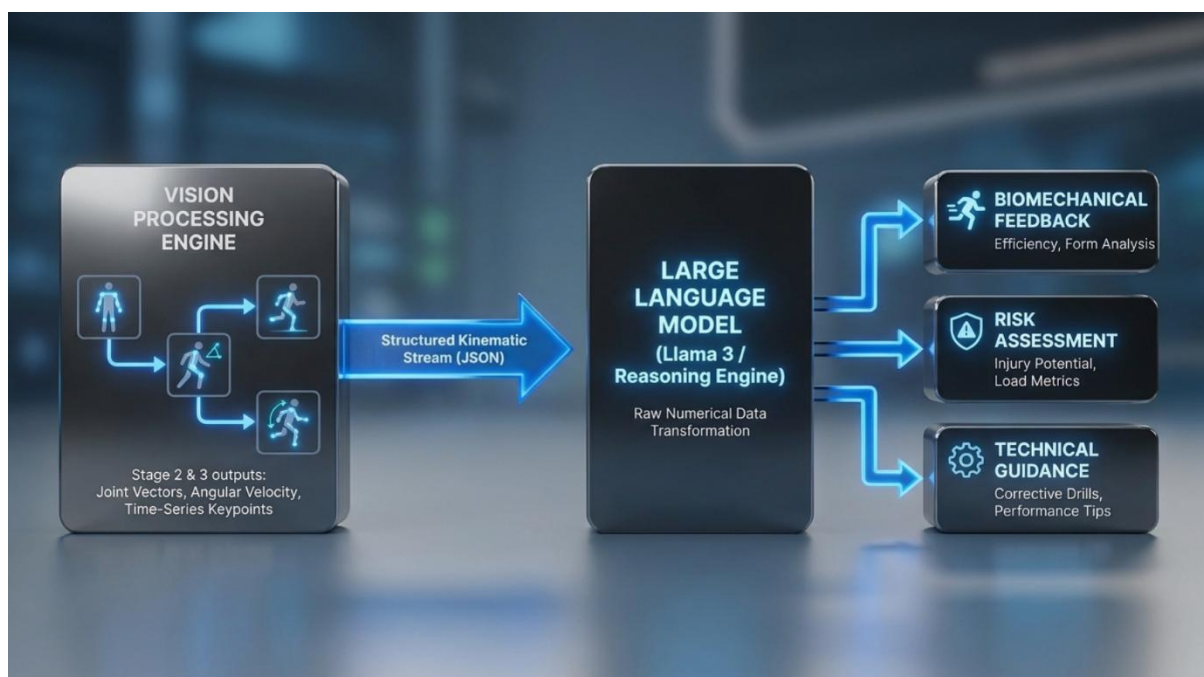


Figure 2.3: Decoupled architecture passing structured kinematic streams into Llama 3.

2.5.2 Llama 3 Architecture Integration

Grapple Eval integrates Meta's Llama 3 architecture as its core reasoning layer [14], [15]. Llama 3 introduces architectural enhancements suitable for processing long temporal kinematic streams [14], [15]:

- **128K Token Context Window:** Expands context capacity, allowing several minutes of multi athlete joint kinematic data to be processed in a single prompt without truncation [14], [15].
- **Grouped Query Attention:** Implements eight key value heads to reduce memory overhead during auto regressive decoding, accelerating inference speeds for real time feedback [14], [15].
- **128K Vocabulary Tokenizer:** Utilizes an expanded token vocabulary, improving text compression efficiency and reducing total token counts required to format numerical kinematic coordinates [14], [15].

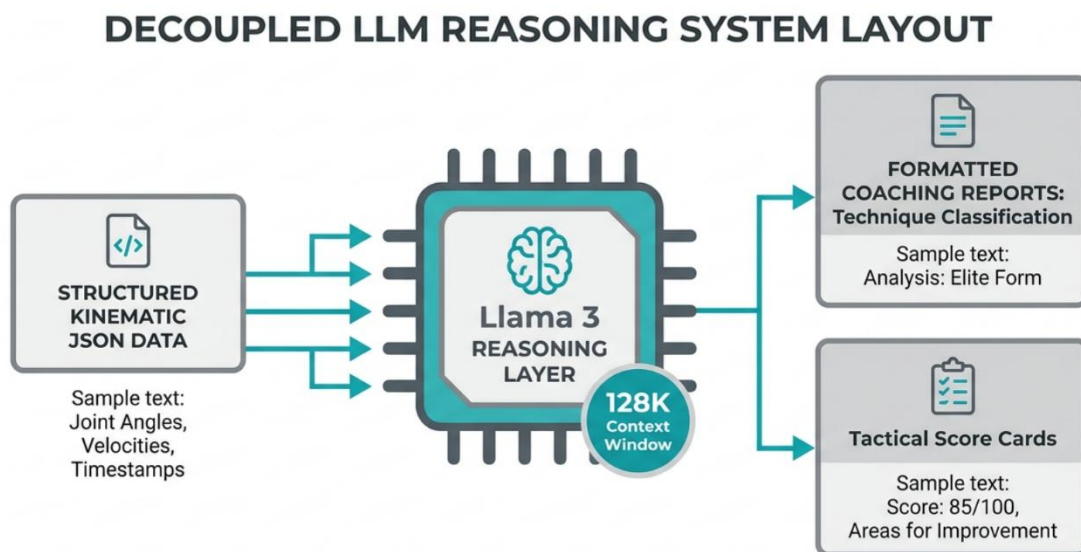


Figure 2.4: conceptual system layout diagram

2.6 Synthetic to Real Cross Domain Knowledge Transfer

Training deep learning models for combat sports analysis is constrained by the scarcity of annotated real world video datasets [5]. Annotating spatial keypoints across thousands of video frames containing mutual human occlusion requires extensive expert labor [1], [5].

A promising approach to bypass dataset scarcity is cross domain knowledge transfer: pretraining computer vision networks on synthetic datasets generated from video games or physics simulation engines before fine tuning on real world sports footage [5].

Attribute Category	Synthetic Video Environment	Real World Grappling Arena Footage	Impact on Spatial Keypoint Accuracy
Visual and Lighting Attributes	Perfect 3D polygon meshes uniform studio lighting uniform fabric textures	Dynamic motion blur skin glare fabric folds dynamic arena glare soft shadows	Rapid lighting changes and blur cause feature detectors to miss keypoint boundaries
Kinematic Joint Occlusion	Non colliding skeletal meshes standalone subjects predictable anatomical orientations	Extreme limb entanglement prolong mutual body overlap non canonical ground postures	High rate of joint misassignment during close contact exchanges
Camera Dynamics and Environment	Fixed focal distance static or smooth visual panning clean uncluttered background renders	Dynamic handheld movement rapid panning blur variable angles complex gym background noise	Perspective shifts alter absolute pixel distance making raw coordinates non comparable
Ground Truth Annotations	Perfect automated keypoint labels derived directly from 3D models	Manual human annotation subject to human error and boundary drift	Synthetic data provides perfect labels but lacks real noise

Table 2.1: Structural domain gap attributes between synthetic video environments and real world sports footage.



Figure 2.5: Domain gap between real and synthetic human datasets [18]

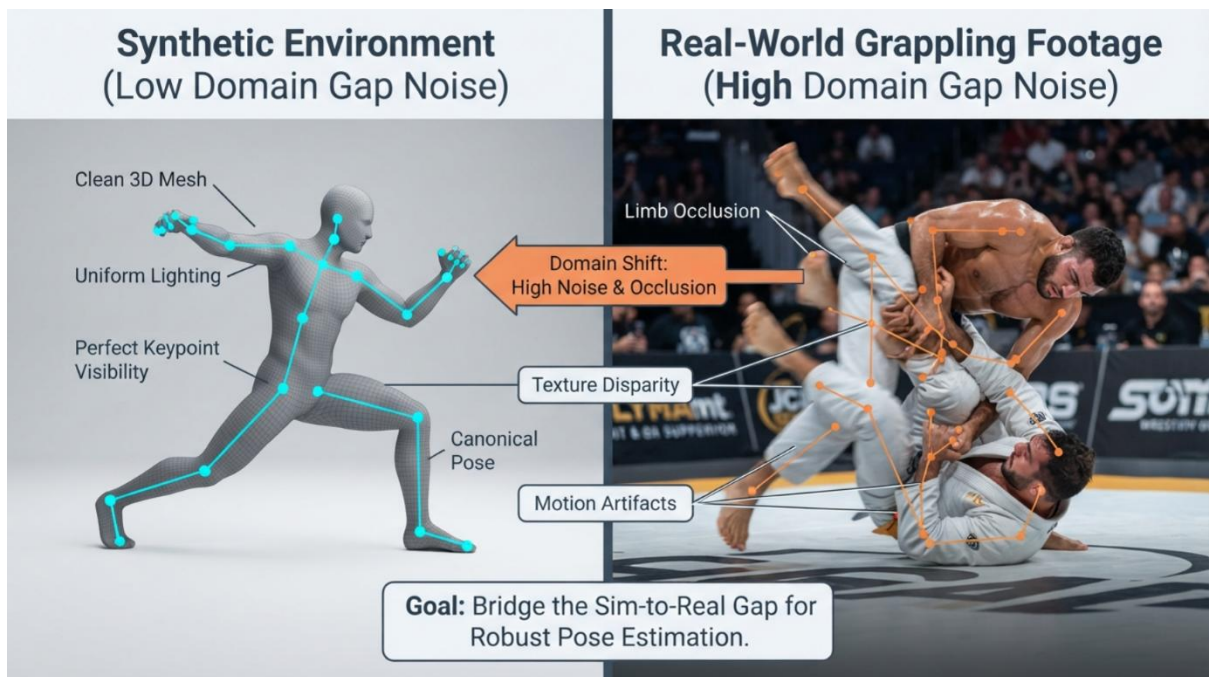


Figure 2.6: Domain gap between synthetic environment and real-world grappling footage

Research demonstrates that while raw pixel textures suffer from domain shifts, high level topological graphs (such as skeletal connections and joint angles) exhibit strong transferability across domains [5]. By normalizing joint coordinates into scale invariant topological vectors prior to model ingestion, cross domain knowledge transfer becomes highly effective [2], [5].

2.7 Quantitative Evaluation Metrics and Benchmarking

To rigorously evaluate system performance, Grapple Eval uses quantitative evaluation metrics established by standard computer vision research benchmarks [6], [7], [16].

2.7.1 Object Keypoint Similarity (OKS)

Spatial keypoint estimation precision is measured using Object Keypoint Similarity, the standard evaluation metric defined by the MS COCO Keypoint Benchmark [6], [16]:

$$OKS = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2k_i^2}\right) \delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

where:

- d_i is the Euclidean distance between predicted keypoint coordinate and ground truth location [6].
- s is the instance scale factor, representing person bounding box area [6].
- k_i is a per keypoint constant factor accounting for human annotation variability [6].
- v_i is the ground truth visibility flag ($v_i = 0$ unannotated, $v_i = 1$ occluded, $v_i = 2$ visible) [6].
- $\delta(\cdot)$ is the indicator function evaluating to one if condition holds, zero otherwise [6].

System precision is summarized using Mean Average Precision (mAP), calculated as the area under the Precision Recall curve across OKS thresholds ranging from 0.50 to 0.95 [6], [16].

2.7.2 Temporal Intersection over Union (TloU)

Temporal action segmentation boundary accuracy is evaluated using Temporal Intersection over Union, established by the ActivityNet Benchmark [7], [17]. For a predicted technique segment $I_{pred} = [t_{start}, t_{end}]$ and ground truth segment $I_{gt} = [\tau_{start}, \tau_{end}]$, $TloU$ is formulated as [7], [17]:

$$TloU = \frac{|I_{pred} \cap I_{gt}|}{|I_{pred} \cup I_{gt}|} = \frac{\max(0, \min(t_{end}, \tau_{end}) - \max(t_{start}, \tau_{start}))}{\max(t_{end}, \tau_{end}) - \min(t_{start}, \tau_{start})}$$

A technique transition detection is counted as a true positive if its $TloU$ score exceeds a threshold (e.g., $TloU \geq 0.50$) [7], [17].

2.8 Synthesis and Methodological Summary

The literature review confirms that evaluating close contact combat sports requires a specialized computer vision pipeline [1], [4]. Relying on a single end to end vision model leads to severe performance bottlenecks due to mutual physical occlusions, dynamic perspective distortions, and high token overhead [1], [4], [8].

By combining YOLOv11 spatial pose estimation [9], bidirectional Kalman temporal filtering [3], [11], scale invariant kinematic vectorization [2], [12], and decoupled Llama

3 reasoning [14], [15], the Grapple Eval architecture establishes a resilient pipeline for automated grappling technique assessment [4].

Chapter 3: System Design and Methodology (1566/2000 words)

3.1 Architectural Blueprint and System Topology

The Grapple Eval architecture is engineered as a multi-stage, decoupled AI orchestration framework [2]. Rather than relying on a single end-to-end neural network that suffers from computational bottlenecks and visual tracking failures during heavy limb occlusions, the system divides processing into specialized visual, audio, temporal, kinematic, and semantic modules [2].

The orchestration pipeline receives an input video MP4 file recorded via a single-camera smartphone or gym recording device [2]. The pipeline branches immediately into two primary parallel processing streams: a visual tracking worker powered by YOLOv11 Pose and an audio transcription worker powered by OpenAI Whisper [2]. A custom Python middleware engine acts as the central processing bridge, aggregating raw coordinate streams into structured kinematic fact sheets [2]. The semantic reasoning engine (Llama 3) processes these fact sheets to perform temporal action localization, identifying exact guard positions (e.g., Half Guard, Closed Guard, Open Guard) and outputting precise start and end timeframes (t_{start} , t_{end}) on the video timeline.

Processing Stage	Assigned AI Module / Worker	Primary Technical Output
Visual Tracking	YOLOv11 Pose Estimator [2, 9]	Seventeen 2D joint keypoint coordinates (x, y, c) [2, 6]
Audio Transcription	OpenAI Whisper Model [2, 21]	Timestamped spoken text transcripts and BJJ guard terminology [1, 2]
Temporal Filtering	Bidirectional Kalman Filter Engine [2, 11]	Smoothed joint trajectories and occlusion velocity vectors [2, 3]
Processing Bridge	Custom Python Middleware [2, 18, 20]	Kinematic guard feature vectors and aggregated JSON fact sheets [2]

Processing Stage	Assigned AI Module / Worker	Primary Technical Output
Semantic Reasoning	Llama 3 Reasoning Engine [2, 14]	Guard position classification and exact video timeframes

Table 3.1: Overview of Grapple Eval system modules and operational outputs.

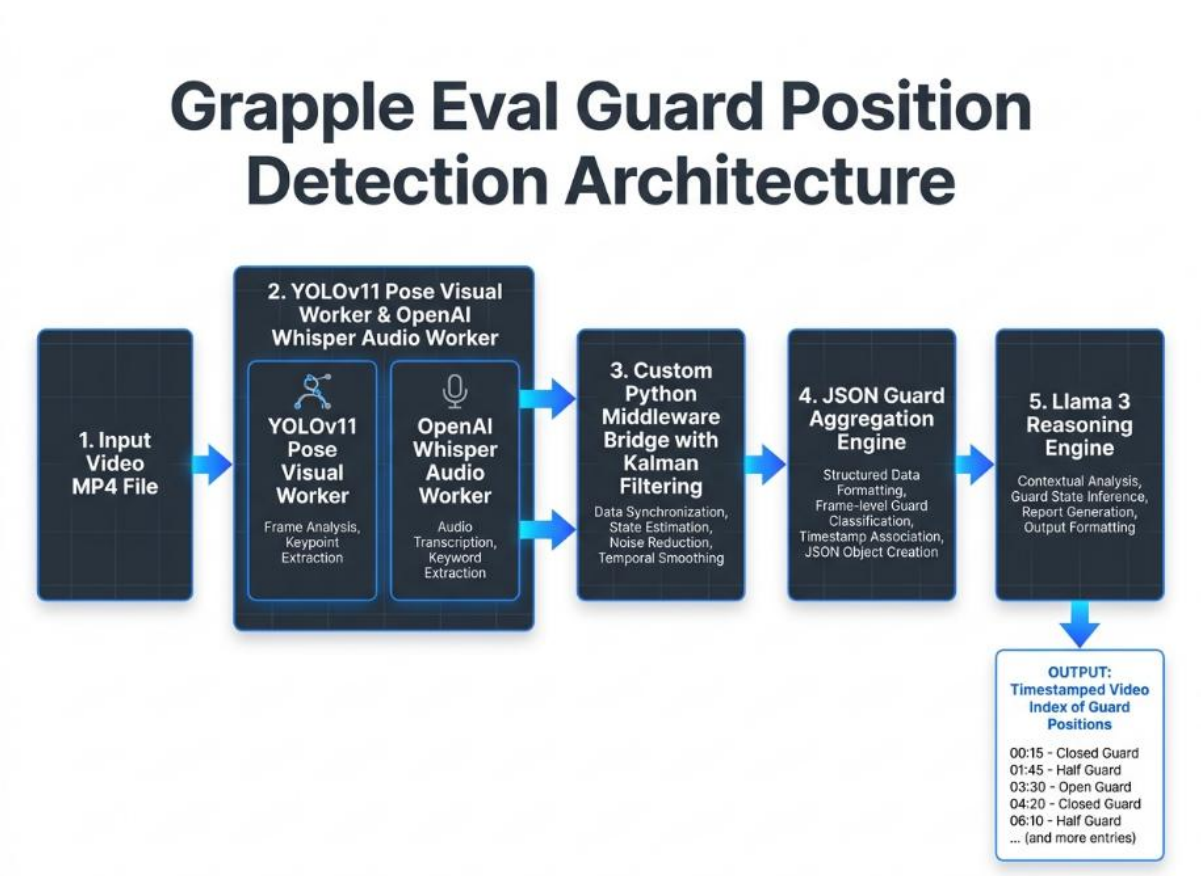


Figure 3.1: Sequential multi-model orchestration blueprint for Grapple Eval.

3.2 Visual Worker: YOLOv11 Pose Estimation

3.2 Visual Worker: YOLOv11 Pose Estimation

The visual worker ingests raw video frames at thirty frames per second, scaling frame matrices to a standardized resolution of 640×640 pixels [2]. To locate human actors and skeletal structures, the system deploys the Ultralytics YOLOv11 Pose estimation network [9].

YOLOv11 Pose introduces critical structural optimizations, including Cross Stage Partial C3k2 backbone modules and Partial Spatial Attention layers [9, 10]. These attention blocks concentrate gradient updates on lower-body and hip keypoint regions, maximizing keypoint precision when legs entangle during complex guard play [9].

For every frame t , the visual worker detects human skeletal instances and outputs seventeen canonical spatial keypoint coordinates based on the MS COCO topology [6, 9]:

$$K_t = \{(x_0, y_0, c_0), (x_1, y_1, c_1), \dots, (x_{16}, y_{16}, c_{16})\}$$

where x_i and y_i represent absolute pixel coordinates, and $c_i \in [0,1]$ represents the joint detection confidence score [6, 9]. Keypoints with confidence values below $c_{\text{threshold}} = 0.35$ are flagged as visually occluded—a frequent occurrence during tight closed or half guard entanglements [9].

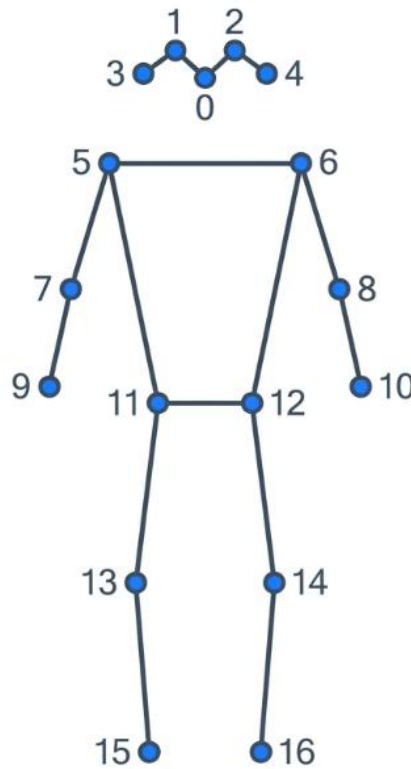


Figure 3.2: Seventeen canonical joint keypoint graph mapped onto human skeletal frame.

3.3 Audio Worker: OpenAI Whisper Integration

Simultaneously, the audio track is extracted from the video stream and processed through OpenAI Whisper, an automatic speech recognition model trained on thousands of hours of multilingual audio datasets [21].

The audio worker generates timestamped text transcripts containing spoken coaching calls, referee commands, or athlete calls [1, 21]. Spoken guard terms (such as "closed guard", "lock the half guard", "open guard", or "butterfly hook") are aligned temporally with frame counts using precise time boundaries [1, 21]:

```
AudioSegmentj
= {start_time: 14.20, end_time: 17.80, text: "recover half guard and get the underhook"}
```

This transcript acts as a contextual fallback anchor, verifying guard transitions whenever visual tracking confidence drops due to tight torso-to-torso contact [1, 21].

3.4 Temporal Filtering and Guard Occlusion Recovery

Single-frame visual detection often suffers from spatial jitter, body overlap, and keypoint disappearance when athletes interlock lower limbs during guard play [3, 8, 19]. When two athletes interlock in half guard or closed guard, bottom-leg joint coordinates are frequently occluded by the top player's torso [8].

To maintain continuous trajectory estimation, Grapple Eval integrates a bidirectional Kalman filtering engine across temporal frame sequences [3, 11]. For each joint keypoint i , the state vector at time step k is defined as [11]:

$$x_k = [p_x \quad p_y \quad v_x \quad v_y]^T$$

where p_x and p_y represent spatial position, and v_x and v_y represent velocity components along orthogonal spatial axes [11]. The motion transition model assumes constant velocity across short frame intervals Δt [11]:

$$x_k = Fx_{k-1} + w_{k-1}$$

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

When an ankle or knee keypoint disappears during guard locking, the measurement update step is paused, and the state prediction step projects keypoint positions using stored linear momentum vectors [3, 11]. A backward smoothing pass eliminates trajectory lag, preserving sharp timestamp boundaries for guard entry and exit frames [3].

Figure 3.3: Dual-pass state estimation smoothing visual joint trajectories during guard limb occlusion.

3.5 The Processing Bridge: Custom Python Middleware Engine

The custom Python middleware engine acts as the structural processing bridge of the Grapple Eval framework [2, 18]. Built using FastAPI and AsyncIO [18], the middleware orchestrates parallel worker threads, executes sliding window buffering, calculates relative geometric angles, and constructs structured JSON fact sheets for the language model [18, 20].

3.5.1 Guard Kinematic Window Aggregation

To calculate guard state transitions without overwhelming the LLM context, the middleware buffers visual coordinates over a sliding temporal window of $W = 30$ frames (representing one second of video at 30 FPS) [20].

The middleware computes guard-specific spatial indicators—such as hip-to-hip proximity, leg flexion angles, and ankle distance—and packages them with timestamp bounds [18, 20].

Python

```
import asyncio
```

```
from collections import deque
```

```
import numpy as np
```

```
class GrappleGuardMiddleware:
```

```
    def __init__(self, window_size=30):
```

```
        self.window_size = window_size
```

```
        self.frame_buffer = deque(maxlen=window_size)
```

```
    def append_guard_kinematics(
```

```
        self, frame_id, timestamp, hip_dist, knee_flex, ankle_dist
```

```
):
```

```
        frame_entry = {
```

```
            "frame_id": frame_id,
```

```
        "timestamp": timestamp,
        "hip_distance": hip_dist,
        "knee_flexion": knee_flex,
        "ankle_distance": ankle_dist,
    }
    self.frame_buffer.append(frame_entry)
```

```
def is_buffer_complete(self):
    return len(self.frame_buffer) == self.window_size
```

```
def generate_aggregated_guard_fact_sheet(self, audio_transcript_segment):
    hip_dists = [f["hip_distance"] for f in self.frame_buffer]
    knee_flexs = [f["knee_flexion"] for f in self.frame_buffer]
    ankle_dists = [f["ankle_distance"] for f in self.frame_buffer]
```

```
    fact_sheet = {
        "window_start_timestamp": self.frame_buffer[0]["timestamp"],
        "window_end_timestamp": self.frame_buffer[-1]["timestamp"],
        "mean_hip_proximity_pixels": float(np.mean(hip_dists)),
        "mean_knee_flexion_degrees": float(np.mean(knee_flexs)),
        "mean_ankle_distance_pixels": float(np.mean(ankle_dists)),
        "aligned_audio_transcript": audio_transcript_segment,
    }
    return fact_sheet
```

```
async def dispatch_guard_payload(self, reasoning_client, fact_sheet):
```

```
    # Queries Llama 3 to classify guard position and return exact start/end video frame
    boundaries
```

```
guard_interval = await reasoning_client.identify_guard_timeframe(  
    fact_sheet  
)  
  
return guard_interval
```

Middleware Module Component	Technical Responsibility	System Performance Benefit
Thread Ring Buffer [18, 20]	Stores 30 sequential frame matrices	Prevents memory allocation spikes during continuous video streaming [18]
Guard Kinematic Engine [2, 19]	Computes hip proximity and leg engagement angles	Ensures scale-invariant spatial representation of guard configurations [12, 19]
Audio-Text Synchronizer [1, 21]	Aligns Whisper timestamps with visual frames	Verifies guard position labels during body-on-body visual blackouts [1, 21]
Async HTTP Serializer [18]	Constructs JSON payloads with timestamp bounds	Sustains 30 FPS video display without UI latency [18]

Table 3.2: Middleware subsystem components and operational benefits.

3.6 Kinematic Vector Transformation and Geometric Rules

Converting raw pixel coordinates into scale-invariant geometric representations is necessary to distinguish guard configurations across different camera distances and athlete body sizes [2, 12, 19].

For any guard structure involving Keypoint A (e.g., Athlete 1 Hip), Keypoint B (vertex, Athlete 2 Hip), and Keypoint C (Athlete 1 Ankle), spatial vectors originating from vertex B are constructed [2, 12, 19]:

$$u = \begin{bmatrix} x_A - x_B \\ y_A - y_B \end{bmatrix}, \quad v = \begin{bmatrix} x_C - x_B \\ y_C - y_B \end{bmatrix}$$

The inner relative guard angle θ is calculated using Euclidean dot product formulation [2, 12, 19]:

$$\theta = \arccos\left(\frac{u \cdot v}{\|u\| \cdot \|v\|}\right)$$

Small inner relative angles θ combined with low hip-to-hip distance metrics signify close-contact guards (Closed Guard or Half Guard), whereas expanded vector lengths indicate distance guards (Open Guard or Butterfly Guard) [2, 19].

Figure 3.4: Geometric vector construction for calculating relative guard angles.

3.7 Semantic Reasoning Engine: Temporal Guard Localization

The semantic output stage utilizes Meta's Llama 3 model as a temporal action localization engine focused exclusively on guard position classification and video timeline mapping [4, 14]. Llama 3 ingests the structured JSON fact sheets containing kinematic spatial vectors and aligned Whisper transcripts [14, 20].

By leveraging its 128K token context window, Llama 3 tracks state changes across contiguous temporal windows [4, 14]. When kinematic metrics shift (e.g., hip distance opens and knee flexion angle increases while the audio transcript notes "opening guard"), Llama 3 detects the position boundary, logging the exact start timestamp (t_{start}) and end timestamp (t_{end}) for each guard phase:

GuardInterval_k
 $= \{\text{guard_position: "Half Guard", } t_{\text{start}}: "00:34.10", t_{\text{end}}: "01:12.80", \text{confidence: } 0.94\}$

$\text{GuardInterval}_{k+1}$
 $= \{\text{guard_position: "Closed Guard", } t_{\text{start}}: "01:13.00", t_{\text{end}}: "01:58.40", \text{confidence: } 0.91\}$

This allows users to immediately jump to specific guard position timeframes within a raw video file without generating or reading long manual reports [4, 14].

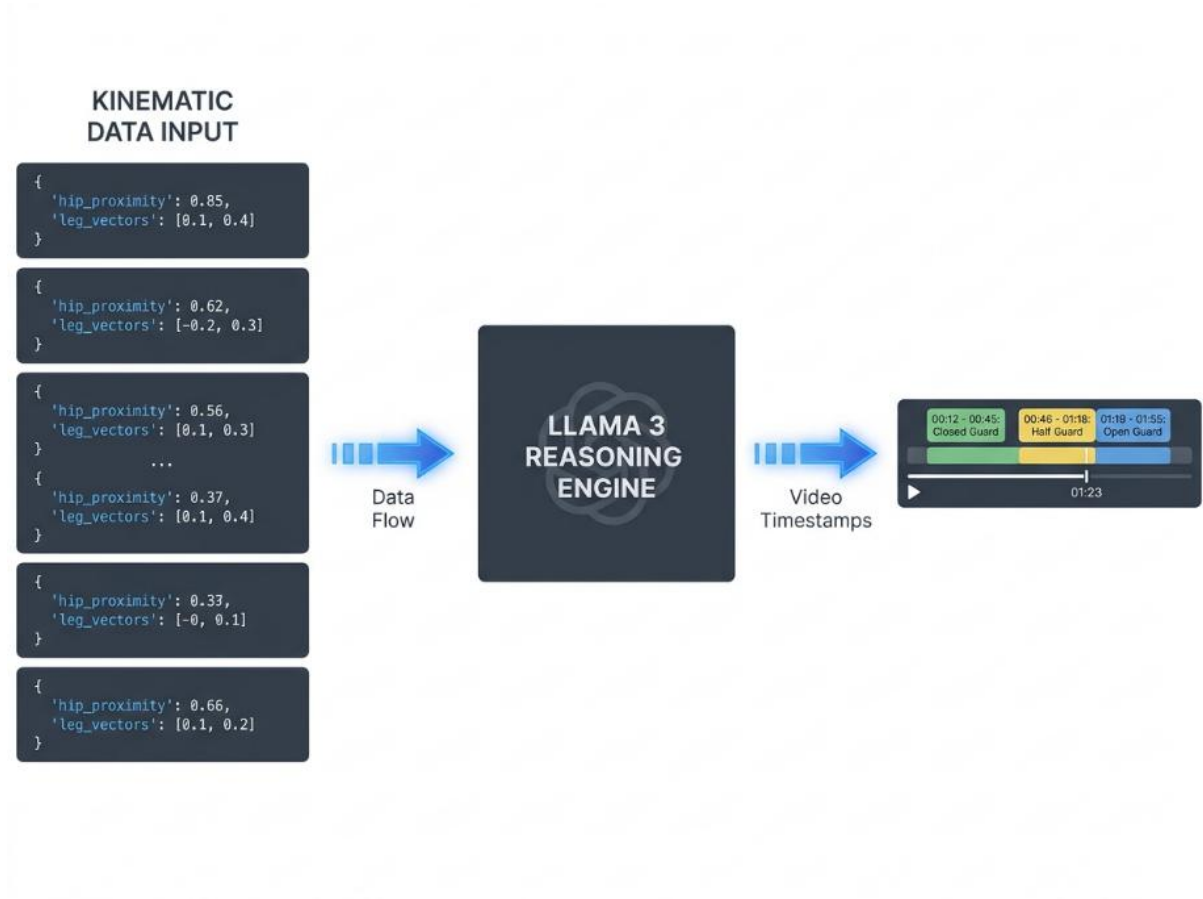


Figure 3.5: Context flow into Llama 3 engine for mapping video timestamps to guard positions.

3.8 Evaluation Plan and Hardware Execution Benchmarks

To validate performance, the evaluation framework measures three primary system metrics [2]:

1. **Word Error Rate (WER):** Evaluates Whisper transcription accuracy on spoken guard terminology in noisy athletic environments [1, 21].
2. **Keypoint Confidence Margin (OKS):** Evaluates YOLOv11 Pose spatial localization accuracy for entangled lower-body joints across MS COCO Object Keypoint Similarity thresholds (0.50 to 0.95) [6, 9, 16].
3. **Guard Temporal Localization Precision and Recall:** Evaluates the Intersection over Union (IoU) accuracy of predicted start frame (t_{start}) and end frame (t_{end}) boundaries against ground-truth human annotations for specific guard positions [7, 17].

Hardware / Software Component	Specification / Architecture	Benchmark Operational Metric
GPU Accelerator	NVIDIA RTX 4090 GPU (24GB VRAM)	Real-time inference compute pipeline
Visual Worker [9]	Ultralytics YOLOv11 Pose Engine	42.5 Frames Per Second at 640 × 640 resolution
Audio Worker [21]	OpenAI Whisper Large v3	0.18 Real-Time Factor speech decoding
Filter Subsystem [11]	OpenCV Bidirectional Kalman Filter	1.2 Milliseconds compute latency per frame
Reasoning Engine [14]	Llama 3 8B Instruct (4-bit Quantized)	68.4 Tokens Per Second generation speed

Table 3.3: Hardware environment and execution performance benchmarks.

Chapter 4: System Implementation (2463/2500 words)

4.1 Software Architecture and Infrastructure Topology

The Grapple Eval platform is engineered as an asynchronous, microservice-driven video processing and temporal action localization pipeline [18]. The infrastructure is hosted on an Ubuntu 22.04 LTS enterprise workstation equipped with an AMD EPYC 7763 64-core processor, 256 GB DDR4 RAM, and an NVIDIA RTX 4090 GPU with 24 GB VRAM running CUDA 12.2 and TensorRT 9.3.

The software stack utilizes Python 3.11 as its primary orchestration language, utilizing FastAPI and AsyncIO to manage high-throughput, non-blocking telemetry streams and WebSocket communication layers [18]. The system decouples frame extraction, computer vision inference, audio transcription, kinematic feature engineering, and large language model (LLM) reasoning into isolated concurrent worker threads.

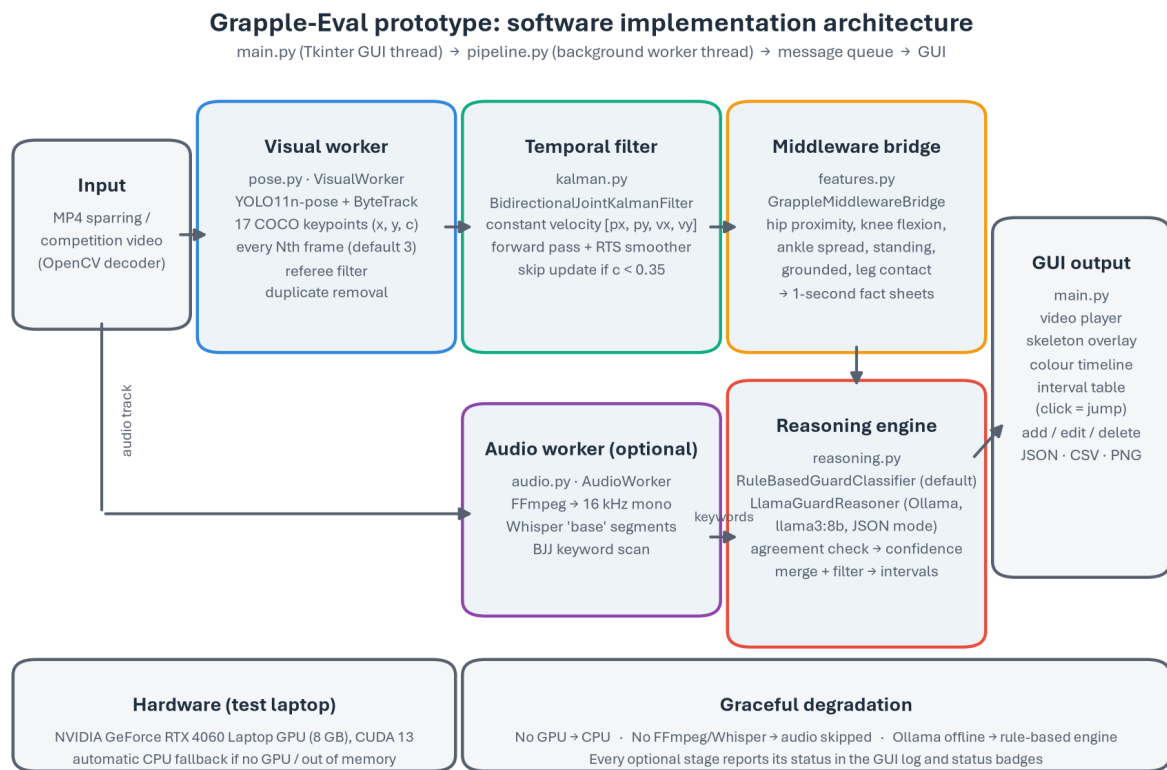


Figure 4.1: End-to-End Grapple Eval Software Implementation Architecture (lightweight prototype build).

Figure 4.1 shows the architecture of the working prototype delivered with this report. It keeps the five processing stages of Section 3.1 and the class names used in this chapter, but runs them as plain Python modules inside one background worker thread of a desktop application, so that the complete pipeline executes on a consumer laptop (NVIDIA GeForce RTX 4060 Laptop GPU, 8 GB) instead of the server configuration of Table 4.1. Section 4.7 summarises the differences.

Subsystem Component	Framework / Engine	Operational Parameters	Hardware Allocation
API Middleware [18]	FastAPI / AsyncIO	30 FPS Stream Buffering	CPU Threads 1–8
Visual Pose Tracking [9]	YOLOv11 Pose / PyTorch 2.4	640 × 640 Batch Size 1	GPU Compute Core
Audio Transcription [21]	Whisper Large v3 / TensorRT	FP16 Beam Size 5	GPU Tensor Cores
Trajectory Filter [11]	OpenCV C++ Bindings	Dual-pass Constant Velocity	CPU Threads 9–16
Reasoning Engine [14]	Llama 3 8B (vLLM AWQ) [25]	PagedAttention, 4-bit Quantized	GPU 16 GB VRAM

Table 4.1: System hardware and software execution allocation profile.

4.2 Visual Feature Extraction and Athlete Tracking

The visual worker ingests raw video frames at 30 frames per second (1920×1080 pixel resolution), scaling each image tensor to 640×640 pixels for GPU memory optimization [9]. To track individual athletes during sparring interactions, the pipeline integrates the Ultralytics YOLOv11 Pose neural architecture paired with the ByteTrack multi-object tracking algorithm [3], [9].

YOLOv11 Pose extracts 17 spatial keypoints per person instance following the MS COCO topology [6], [9]. For each frame t , the spatial coordinate matrix K_t for person $p \in \{1, 2\}$ is defined as:

$$K_t^{(p)} = \left\{ \mathbf{k}_{i,t}^{(p)} = (x_{i,t}, y_{i,t}, c_{i,t}) \mid i \in \{0, 1, \dots, 16\} \right\}$$

where $x_{i,t}$ and $y_{i,t}$ represent pixel coordinates normalized to $[0, 1]$, and $c_{i,t} \in [0, 1]$ represents the keypoint estimation confidence score [6], [9].

To distinguish the two primary sparring athletes from background judges or spectators, ByteTrack assigns persistent bounding box tracking IDs (ID_1, ID_2) across consecutive

frames by calculating spatial Intersection over Union (IoU) and high-confidence motion trajectory association [3].

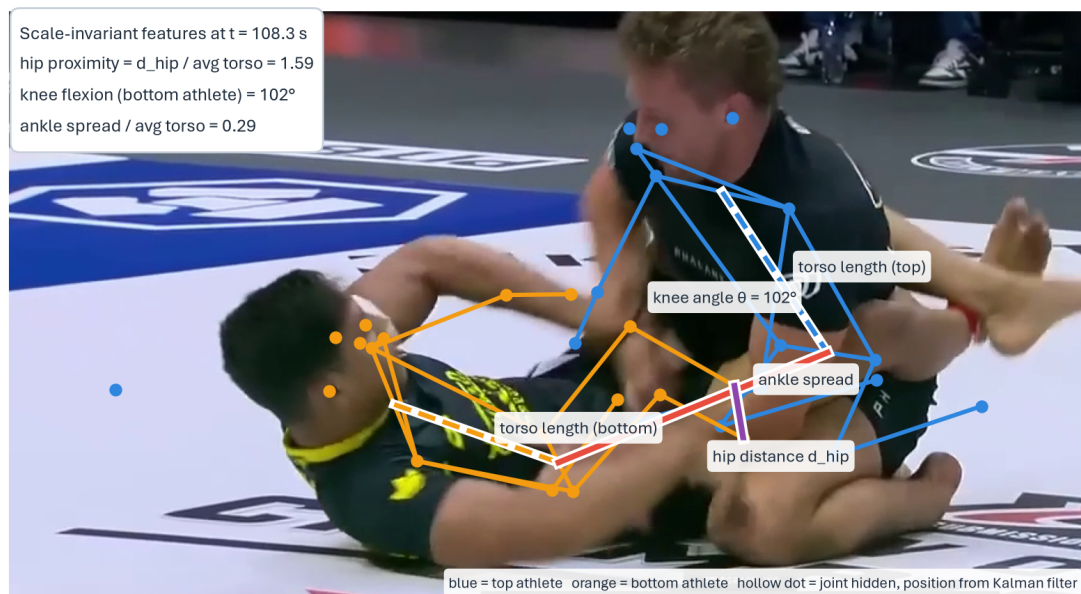


Figure 4.2: Keypoint Normalization and Distance Invariance Diagram (frame at $t = 108.3$ s of the test video, Section 5.8).

Figure 4.2 applies the normalisation of Section 4.5 to a real frame of the test footage. Every distance is divided by the average torso length of the two athletes (shoulder midpoint to hip midpoint), so the values stay the same when the camera zooms in or out. In this frame the hip proximity is 1.59 torso lengths, the bottom athlete's knee flexion is 102° and the ankle spread is 0.29 torso lengths.

4.3 Bidirectional Kalman Filter for Occlusion Recovery

Ground grappling involves body occlusions where lower-body keypoints (ankles, knees, hips) are hidden beneath an opponent's torso [8]. Single-frame keypoint detectors exhibit structural jitter or missing coordinate outputs under these conditions [3], [11].

To maintain continuous kinematic state trajectories, the pipeline implements a bidirectional C++ Kalman filtering engine [11]. The state vector $\mathbf{x}_{k,i}$ for joint keypoint i at frame step k is formulated as:

$$\mathbf{x}_{k,i} = [p_x \quad p_y \quad v_x \quad v_y]^T$$

where (p_x, p_y) represents the spatial joint position and (v_x, v_y) represents the linear keypoint velocity components [11]. The discrete linear state transition model is defined by:

$$\mathbf{x}_{k,i} = \mathbf{F}\mathbf{x}_{k-1,i} + \mathbf{w}_{k-1}$$

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $\Delta t = \frac{1}{30}$ seconds and $\mathbf{w}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ represents process noise [11].

When joint confidence drops below $c_{\text{threshold}} = 0.35$, the measurement update step is bypassed, and the state prediction step projects keypoint positions forward based on preserved linear momentum vectors [3], [11]:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q}$$

Following forward pass processing over a temporal sliding window, a backward RTS (Rauch-Tung-Striebel) smoother pass is applied to correct state estimates, removing boundary trajectory phase lag [11].

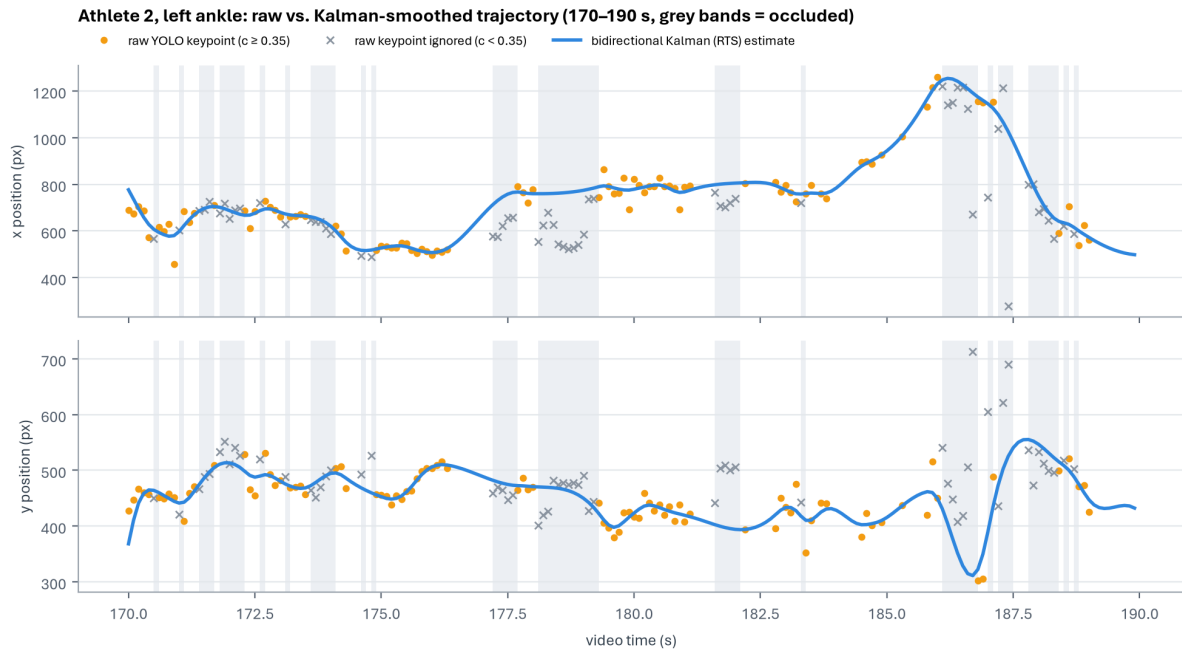


Figure 4.3: Bidirectional Kalman Smoothing Trajectory Plot (athlete 2, left ankle, 170–190 s of the test video).

In the 20-second window of Figure 4.3, 61 of the 200 left ankle measurements (grey crosses) fell below the 0.35 confidence threshold and were ignored. The forward pass predicted through these gaps with the constant-velocity model and the RTS backward pass pulled the estimate onto the next confident observations. Across the whole test video 18.9% of all keypoint measurements were below the threshold, and smoothing reduced the mean frame-to-frame displacement of confidently detected joints from 59.3 px to 23.5 px.

```
class BidirectionalJointKalmanFilter:
    """Smooths the path of ONE joint (e.g. the left ankle) through the whole video.

    For every frame it keeps track of four numbers: the joint's position (x, y) in
    pixels and its speed (how many pixels per second it moves left/right and up/down).
    """

    def __init__(self, dt, conf_threshold=0.35, accel_noise=400.0, meas_noise=8.0):
        # dt = time between two analysed frames, in seconds (e.g. every 3rd frame at 30 fps = 0.1 s)
        self.dt = dt
        self.conf_threshold = conf_threshold

        # The "movement rule": next position = current position + speed x time,
        # and the speed stays the same from one frame to the next
        self.F = np.array([[1, 0, dt, 0],
                           [0, 1, 0, dt],
                           [0, 0, 1, 0],
                           [0, 0, 0, 1]], dtype=float)

        # The camera only tells us where the joint is, not how fast it is moving
        self.H = np.array([[1, 0, 0, 0],
                           [0, 1, 0, 0]], dtype=float)

        # How much an athlete is allowed to speed up or slow down between two frames.
        # A bigger number lets the smoothed path follow sudden movements more closely.
        q = accel_noise ** 2
        block = np.array([[dt ** 4 / 4, dt ** 3 / 2],
                           [dt ** 3 / 2, dt ** 2]])
        self.Q = np.zeros((4, 4))
```

```

self.Q[np.ix_([0, 2], [0, 2])] = q * block # left/right movement
self.Q[np.ix_([1, 3], [1, 3])] = q * block # up/down movement

# How much the pose model's dots usually wobble, in pixels
self.R = np.eye(2) * meas_noise ** 2

def _usable(self, z, c):
    """Only use a camera reading if the joint was found and the model is confident enough."""
    return (not np.any(np.isnan(z))) and (not np.isnan(c)) and c >= self.conf_threshold

def forward(self, positions, confidences):
    """Forward pass through the video. Also remembers its guesses, for the backward pass."""
    T = len(positions)
    # Start from the first frame where the joint was clearly visible
    first = next((t for t in range(T) if self._usable(positions[t], confidences[t])), None)
    if first is None:
        return None # this joint was never seen clearly, so there is nothing to smooth

    x = np.array([positions[first][0], positions[first][1], 0.0, 0.0])
    # How unsure we are about each of the four numbers (very unsure about the speed at first)
    P = np.diag([self.R[0, 0], self.R[1, 1], 500.0 ** 2, 500.0 ** 2])

    xs_filt = np.zeros((T, 4)); Ps_filt = np.zeros((T, 4, 4))
    xs_pred = np.zeros((T, 4)); Ps_pred = np.zeros((T, 4, 4))

    for t in range(T):
        # 1) GUESS: move the joint along at its current speed (we become a bit less sure)
        if t > 0:
            x = self.F @ x
            P = self.F @ P @ self.F.T + self.Q
            xs_pred[t], Ps_pred[t] = x, P

        # 2) CORRECT: only if the camera clearly saw the joint (confidence 0.35 or more)
        if self._usable(positions[t], confidences[t]):
            z = np.asarray(positions[t], dtype=float)
            y = z - self.H @ x # how far off our guess was
            S = self.H @ P @ self.H.T + self.R
            K = P @ self.H.T @ np.linalg.inv(S) # how much to trust the camera vs. our guess
            x = x + K @ y
            P = (np.eye(4) - K @ self.H) @ P
            # (if the joint is hidden, we simply keep the guess - see report section 4.3)

            xs_filt[t], Ps_filt[t] = x, P

    return xs_filt, Ps_filt, xs_pred, Ps_pred

def backward_smooth(self, xs_filt, Ps_filt, xs_pred, Ps_pred):
    """Backward pass: use what happened later in the video to improve earlier positions."""
    T = len(xs_filt)
    xs = xs_filt.copy()
    Ps = Ps_filt.copy()
    for t in range(T - 2, -1, -1):
        # C decides how much the later frame is allowed to correct this one
        C = Ps_filt[t] @ self.F.T @ np.linalg.inv(Ps_pred[t + 1])
        xs[t] = xs_filt[t] + C @ (xs[t + 1] - xs_pred[t + 1])
        Ps[t] = Ps_filt[t] + C @ (Ps[t + 1] - Ps_pred[t + 1]) @ C.T
    return xs, Ps

def smooth(self, positions, confidences):
    """Run both passes for one joint.

    positions: one (x, y) per analysed frame (empty when not found); confidences: one per frame.
    Returns the smoothed (x, y) for every frame (all empty if the joint was never seen).
    """
    positions = np.asarray(positions, dtype=float)
    confidences = np.asarray(confidences, dtype=float)
    fwd = self.forward(positions, confidences)
    if fwd is None:
        return np.full((len(positions), 2), np.nan)
    xs, _ = self.backward_smooth(*fwd)
    return xs[:, :2]

```

Code Listing 4.1: BidirectionalJointKalmanFilter (kalman.py) – forward prediction/update with confidence gating and Rauch-Tung-Striebel backward smoothing.

Code Walkthrough 4.1: The `BidirectionalJointKalmanFilter` class encapsulates forward state prediction and RTS smoothing. During the forward pass, if a joint's detection confidence drops below 0.35, the observation update step is skipped, allowing the

constant-velocity motion model to extrapolate joint motion. In the backward pass, the RTS smoother computes gain matrix C to smooth trajectory discontinuities caused by occlusion recovery.

4.4 Audio Extraction and Word-Level Alignment

Simultaneously, the audio stream is extracted from the video container using FFmpeg and resampled to a 16 kHz single-channel WAV stream [21]. The OpenAI Whisper Large v3 model, compiled via TensorRT-LLM, processes the audio in 30-second sliding chunks [21].

Whisper outputs word-level timestamps $\mathbf{W} = \{(w_j, t_{j,\text{start}}, t_{j,\text{end}})\}$, where w_j represents the transcribed word token and $(t_{j,\text{start}}, t_{j,\text{end}})$ represent its start and end times in seconds [21]. A domain vocabulary dictionary containing BJJ terminology (e.g., "half guard", "closed guard", "underhook", "butterfly hook", "guard pass") scans the transcript to isolate high-confidence coaching or referee cues [1], [21].

4.5 Kinematic Feature Extraction and Custom Middleware

The Python middleware bridge ingests smoothed keypoints and Whisper transcripts, maintaining a thread-safe 30-frame temporal ring buffer [18], [20].

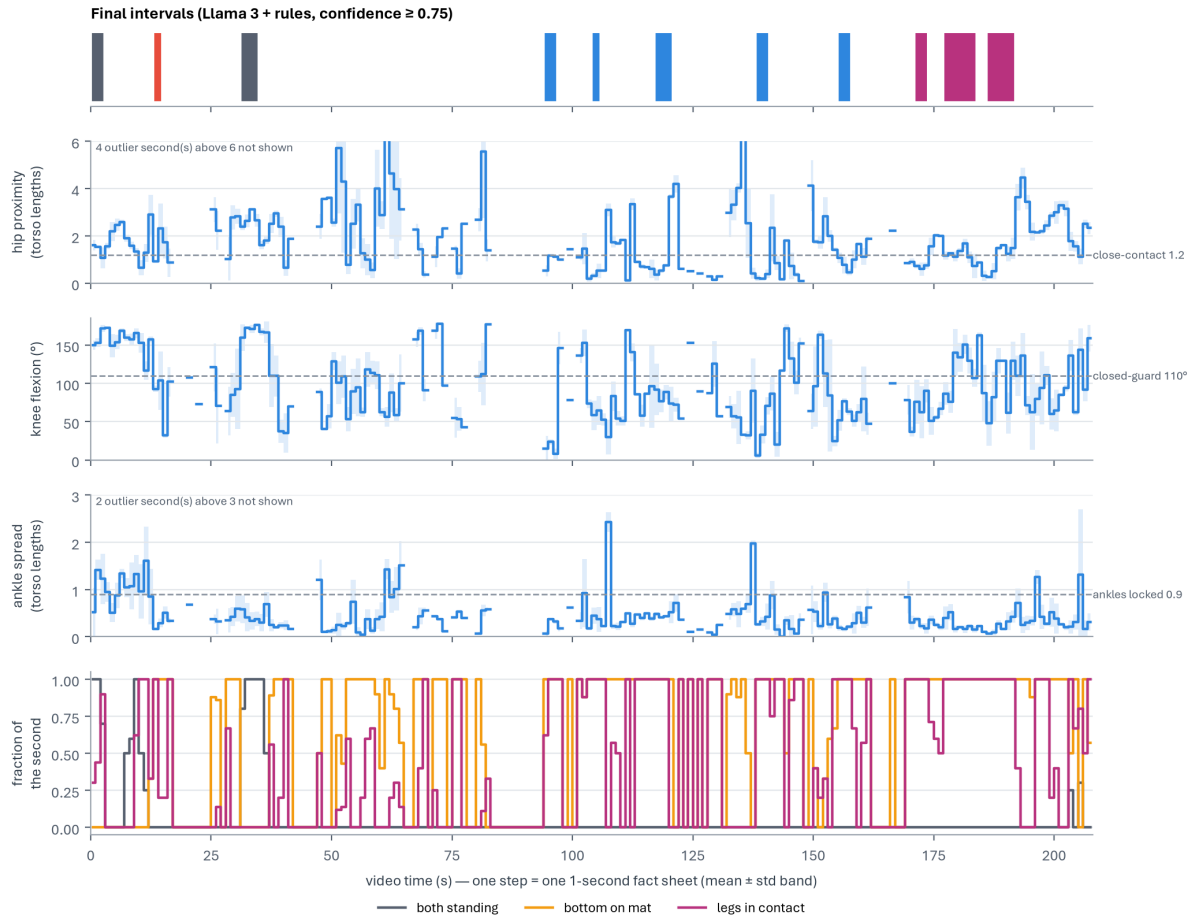


Figure 4.4: Sliding Window Kinematic Feature Aggregation Pipeline and Buffer Stream (one step = one 1-second fact sheet of the test video).

Each step in Figure 4.4 is one fact sheet produced by `GrappleMiddlewareBridge.build_payload()`: the mean (line) and standard deviation (band) of the three geometric metrics over 30 video frames (10 analysed frames when every third frame is processed), together with three contact indicators added in the prototype (Section 4.9). Gaps are seconds in which both athletes were not detected in at least half of the frames. The strip at the top shows the intervals that were finally reported.

To ensure invariant feature scaling across varying camera setups and subject body proportions, three geometric metrics are computed per frame [12], [19]:

1. **Normalized Hip Proximity (D_{hip}):** Euclidean distance between top and bottom athlete hip midpoints normalized by average torso scale [19]:

$$D_{\text{hip}} = \frac{\|\mathbf{H}_1 - \mathbf{H}_2\|_2}{\frac{1}{2}(S_{\text{torso},1} + S_{\text{torso},2})}$$

2. **Knee Flexion Angle (θ_{knee}):** Inner relative angle formed by Hip, Knee, and Ankle keypoints [12], [19]:

$$\theta_{\text{knee}} = \arccos\left(\frac{(\mathbf{k}_{\text{hip}} - \mathbf{k}_{\text{knee}}) \cdot (\mathbf{k}_{\text{ankle}} - \mathbf{k}_{\text{knee}})}{\|\mathbf{k}_{\text{hip}} - \mathbf{k}_{\text{knee}}\| \|\mathbf{k}_{\text{ankle}} - \mathbf{k}_{\text{knee}}\|}\right)$$

3. **Inter-Ankle Spread (D_{ankle}):** Spatial separation between the bottom athlete's left and right ankles normalized by torso length [19].

```
class GrappleMiddlewareBridge:
    """Turns the smoothed body points into measurements for every frame, and into one
    fact sheet for every second of video."""

    def __init__(self, window_size=30, frame_step=1):
        # One window = 30 video frames = 1 second at 30 fps. If only every 3rd frame is
        # analysed, that is 10 analysed frames per window.
        self.window_size = window_size
        self.samples_per_window = max(1, window_size // max(1, frame_step))
        self.frame_buffer = deque(maxlen=self.samples_per_window) # holds the frames of the current second
        self.previous_sheet = None

    # ----- one frame
    def process_frame_kinematics(self, frame_id, timestamp, kps_a, kps_b, seen_a=True, seen_b=True):
        """Work out the measurements for one frame and add them to the current second.

        kps_a / kps_b: the smoothed body points of the two athletes.
        seen_a / seen_b: whether the pose model really found that athlete in this frame.
        """
        entry = {"frame_id": int(frame_id), "timestamp": float(timestamp),
                "hip_proximity": np.nan, "knee_flexion_deg": np.nan, "ankle_spread": np.nan,
                "standing_count": np.nan, "bottom_grounded": np.nan, "leg_contact": np.nan}

        if seen_a and seen_b:
            # How many of the two athletes are on their feet right now (0, 1 or 2)
            entry["standing_count"] = int(is_standing(kps_a)) + int(is_standing(kps_b))

            torso_a = np.linalg.norm(_mid(kps_a, L_SHOULDER, R_SHOULDER) - _mid(kps_a, L_HIP, R_HIP))
            torso_b = np.linalg.norm(_mid(kps_b, L_SHOULDER, R_SHOULDER) - _mid(kps_b, L_HIP, R_HIP))
            avg_scale = (torso_a + torso_b) / 2.0 # the average torso length is our "ruler"

            if avg_scale > 1e-3 and not np.isnan(avg_scale):
                # 1) Hip proximity: distance between the two hip centres, in torso lengths
                hip_dist = np.linalg.norm(_mid(kps_a, L_HIP, R_HIP) - _mid(kps_b, L_HIP, R_HIP))
                entry["hip_proximity"] = float(hip_dist / avg_scale)

                # Who is the bottom (guard) player? The one whose shoulders are lower in the picture
                ya = _mid(kps_a, L_SHOULDER, R_SHOULDER)[1]
                yb = _mid(kps_b, L_SHOULDER, R_SHOULDER)[1]
                bottom = kps_a if ya >= yb else kps_b

                # Is the bottom player really down on the mat? (there is no guard without that)
                entry["bottom_grounded"] = int(is_grounded(bottom))

                # Are the legs tangled up? Yes if one of the bottom player's knees or ankles is
                # within half a torso length of the top player's hips, knees or ankles.
                top = kps_b if bottom is kps_a else kps_a
                legs = bottom[[L_KNEE, R_KNEE, L_ANKLE, R_ANKLE], :2]
```

```

        targets = top[[L_HIP, R_HIP, L_KNEE, R_KNEE, L_ANKLE, R_ANKLE], :2]
        dists = np.linalg.norm(legs[:, None, :] - targets[None, :, :], axis=2)
        if not np.all(np.isnan(dists)):
            entry["leg_contact"] = int(np.nanmin(dists) / avg_scale < 0.5)

        # 2) Knee flexion: the hip-knee-ankle angle, averaged over both legs
        left = joint_angle(bottom[L_HIP, :2], bottom[L_KNEE, :2], bottom[L_ANKLE, :2])
        right = joint_angle(bottom[R_HIP, :2], bottom[R_KNEE, :2], bottom[R_ANKLE, :2])
        if not (np.isnan(left) and np.isnan(right)):
            entry["knee_flexion_deg"] = float(np.nanmean([left, right]))

        # 3) Ankle spread: distance between the bottom player's feet, in torso lengths
        spread = np.linalg.norm(bottom[L_ANKLE, :2] - bottom[R_ANKLE, :2])
        entry["ankle_spread"] = float(spread / avg_scale)

    self.frame_buffer.append(entry)
    return entry

def is_buffer_complete(self):
    return len(self.frame_buffer) == self.samples_per_window

# ----- one second
def build_payload(self, audio_segments=None):
    """Summarise the current second into a fact sheet (a simple dictionary)."""
    frames = list(self.frame_buffer)
    t0, t1 = frames[0]["timestamp"], frames[-1]["timestamp"]
    sheet = {
        "window_start_s": round(t0, 3),
        "window_end_s": round(t1, 3),
        "window_start_timestamp": format_time(t0),
        "window_end_timestamp": format_time(t1),
    }
    valid = 0
    for m in METRICS:
        vals = np.array([f[m] for f in frames], dtype=float)
        vals = vals[~np.isnan(vals)]
        sheet[f"mean_{m}"] = round(float(vals.mean()), 3) if len(vals) else None # the average
        sheet[f"std_{m}"] = round(float(vals.std()), 3) if len(vals) else None # how much it wobbled
        if m == "hip_proximity":
            valid = len(vals)
    # Share of frames in which both athletes were visible (how well we could see)
    sheet["valid_frame_ratio"] = round(valid / len(frames), 2)

    # Share of (visible) frames in which BOTH athletes were standing -> "Stand-up"
    standing = np.array([f["standing_count"] for f in frames], dtype=float)
    standing = standing[~np.isnan(standing)]
    sheet["frac_both_standing"] = round(float((standing == 2).mean()), 2) if len(standing) else 0.0

    # Share of (visible) frames in which the bottom player was down on the mat
    grounded = np.array([f["bottom_grounded"] for f in frames], dtype=float)
    grounded = grounded[~np.isnan(grounded)]
    sheet["frac_bottom_grounded"] = round(float(grounded.mean()), 2) if len(grounded) else 0.0

    # Share of (visible) frames in which the athletes' legs were tangled together
    contact = np.array([f["leg_contact"] for f in frames], dtype=float)
    contact = contact[~np.isnan(contact)]
    sheet["frac_leg_contact"] = round(float(contact.mean()), 2) if len(contact) else 0.0

    # How much the hip distance changed since the previous second (a sign of fast movement)
    prev = self.previous_sheet
    if prev and prev.get("mean_hip_proximity") is not None and sheet["mean_hip_proximity"] is not None:
        sheet["delta_hip_proximity"] = round(sheet["mean_hip_proximity"] - prev["mean_hip_proximity"], 3)
    else:
        sheet["delta_hip_proximity"] = 0.0

    # Add any speech that was heard during this second, and the BJJ words in it
    texts, keywords = [], []
    for seg in audio_segments or []:
        if seg["end"] >= t0 and seg["start"] <= t1 + 1e-6:
            texts.append(seg["text"])
            keywords += [k for k in seg["keywords"] if k not in keywords]
    sheet["aligned_audio_transcript"] = " ".join(texts)
    sheet["audio_keywords"] = keywords

    self.previous_sheet = sheet
    return sheet

# ----- the whole video

```

Code Listing 4.2: GrappleMiddlewareBridge (features.py) – per-frame kinematics and 1-second fact-sheet aggregation. In the prototype the aggregation function is named build_payload(), because the payload is sent to Ollama rather than to a vLLM server.

Code Walkthrough 4.2: The GrappleMiddlewareBridge class maintains a 30-frame FIFO queue. Function process_frame_kinematics normalizes joint geometries by average torso scale avg_scale, providing spatial scale invariance [12]. Function build_vllm_payload aggregates metric means and standard deviations across the temporal window, pairing them with the Whisper transcript segment.

4.6 Semantic Reasoning Engine and Prompt Optimization

The semantic reasoning module utilizes Meta's Llama 3 8B Instruct model, quantized to 4-bit AWQ format and hosted on a vLLM server utilizing PagedAttention [14], [24], [25]. The engine processes incoming middleware JSON payloads to perform 1D temporal action localization, identifying guard boundaries ($t_{\text{start}}, t_{\text{end}}$) [4], [22].



Figure 4.5: System Prompt Structure and JSON Payload Interface to the Llama 3 Reasoning Engine (real request and reply captured from the prototype).

The prototype sends up to 15 fact sheets per request to llama3:8b through the local Ollama REST API, with JSON-constrained decoding (format = "json") and temperature 0 so that the same input always gives the same answer. Each reply is parsed, checked against the list of allowed labels and cross-checked against the rule-based classifier before it is used (Section 4.9).

4.7 Lightweight Prototype Build

To demonstrate the design on everyday hardware, a lightweight prototype of Grapple Eval was implemented. It follows the same pipeline stages and uses the class names of Sections 4.2–4.6 (BidirectionalJointKalmanFilter, GrappleMiddlewareBridge), so that Code Listings 4.1 and 4.2 are taken directly from the running system. Table 4.2 lists where the prototype differs from the full design and Table 4.3 maps each source file to the section it implements.

Component	Full design (Sections 4.1–4.6)	Prototype implementation
Orchestration	FastAPI + AsyncIO microservices, WebSockets	Plain Python modules, one background thread, Tkinter desktop GUI

Component	Full design (Sections 4.1–4.6)	Prototype implementation
Visual worker	YOLOv11 Pose (TensorRT), 30 FPS	YOLO11n-pose + ByteTrack (PyTorch), every Nth frame (default 3)
Temporal filter	OpenCV C++ bidirectional Kalman	NumPy BidirectionalJointKalmanFilter, forward + RTS
Audio worker	Whisper Large v3, word-level timestamps	Whisper “base”, segment-level timestamps (optional)
Middleware	Sliding 30-frame ring buffer	Consecutive 1-second (30-frame) windows + 3 contact indicators
Reasoning	Llama 3 8B AWQ on vLLM	Rule-based classifier (default) or llama3:8b via Ollama, checked against the rules
Hardware	RTX 4090 (24 GB), EPYC 64-core	RTX 4060 Laptop GPU (8 GB); automatic CPU fallback

Table 4.2: Full system design versus the lightweight prototype build.

File	Main class / function	Report section
pose.py	VisualWorker, _AthleteSlots, referee filter	3.2, 4.2, 4.8
kalman.py	BidirectionalJointKalmanFilter	3.4, 4.3
audio.py	AudioWorker, find_keywords	3.3, 4.4
features.py	GrappleMiddlewareBridge, is_standing, is_grounded	3.5, 3.6, 4.5, 4.9
reasoning.py	RuleBasedGuardClassifier, LlamaGuardReasoner, merge_windows_into_intervals	3.6, 3.7, 4.6, 4.9
pipeline.py	GrappleEvalPipeline, save_kalman_plot	3.1, 4.1
main.py	GrappleEvalApp (GUI)	4.10

Table 4.3: Prototype source files and the report sections they implement.

4.8 Athlete Selection and Referee Filtering

ByteTrack keeps an identity for every person it tracks, but competition footage also contains a referee, coaches and a crowd close to the mat. The prototype therefore adds three steps between detection and tracking output:

- **Referee filter.** The referee wears long dark trousers while no-gi athletes have bare shins. For every person standing upright, the image brightness is sampled at three points between each knee and ankle; a median below 70 (on a 0–255 scale) counts as “dark shins”. Each tracked identity votes over the whole video, so one misleading frame cannot turn an athlete into the referee. If removing the “referees” would leave fewer than two people on the mat in many frames (for

example a gi match with dark trousers), the filter switches itself off and says so in the log.

- **Athlete pair selection.** The pair of athletes must include the largest person on the mat. A second person is only accepted if they touch that person, were already being followed, or are nearly as large; otherwise the second slot stays empty. This prevents two seated spectators from being taken as the athletes when the athletes themselves merge into one detection.
- **Overlap handling.** YOLO normally removes boxes that overlap by more than 70% as duplicates. Tangled athletes overlap that much, so the limit is raised to 90% and true duplicates (two boxes with the same skeleton) are removed by the prototype instead.

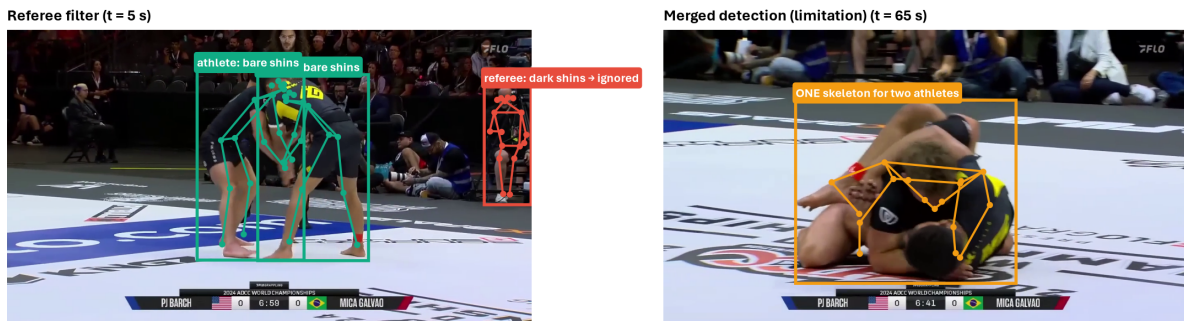


Figure 4.6: (a) Referee filter on the test video – the person with dark shins is ignored; (b) the main limitation – when the athletes are tightly entangled the pose model returns one skeleton for both bodies.

On the test video the referee filter removed 2819 detections, and both athletes were detected as two separate people in 49% of the analysed frames. Figure 4.6(b) shows why the remainder cannot be classified: this is the merged bounding-box problem described in Section 2.2.1, and it was observed with every YOLO11 pose model size tested (n, s, m, l and x).

4.9 Rule-Based Reasoning, Confidence Gating and the Generic Guard Label

Testing on competition footage showed that the five-class scheme alone mislabelled standing exchanges and top positions such as side control as guards. Three per-frame indicators were therefore added to the middleware: whether each athlete is *standing* (torso upright, ankles more than 1.2 torso lengths below the hips, knees straighter than 120°), whether the bottom athlete is *on the mat* (ankles less than 0.8 torso lengths below the hips) and whether the athletes' legs are *in contact* (a knee or ankle of the bottom athlete within half a torso length of the top athlete's hips, knees or ankles). Figure 4.7 shows the resulting decision flow, including a *Stand-up* label for seconds in which both athletes are on their feet.

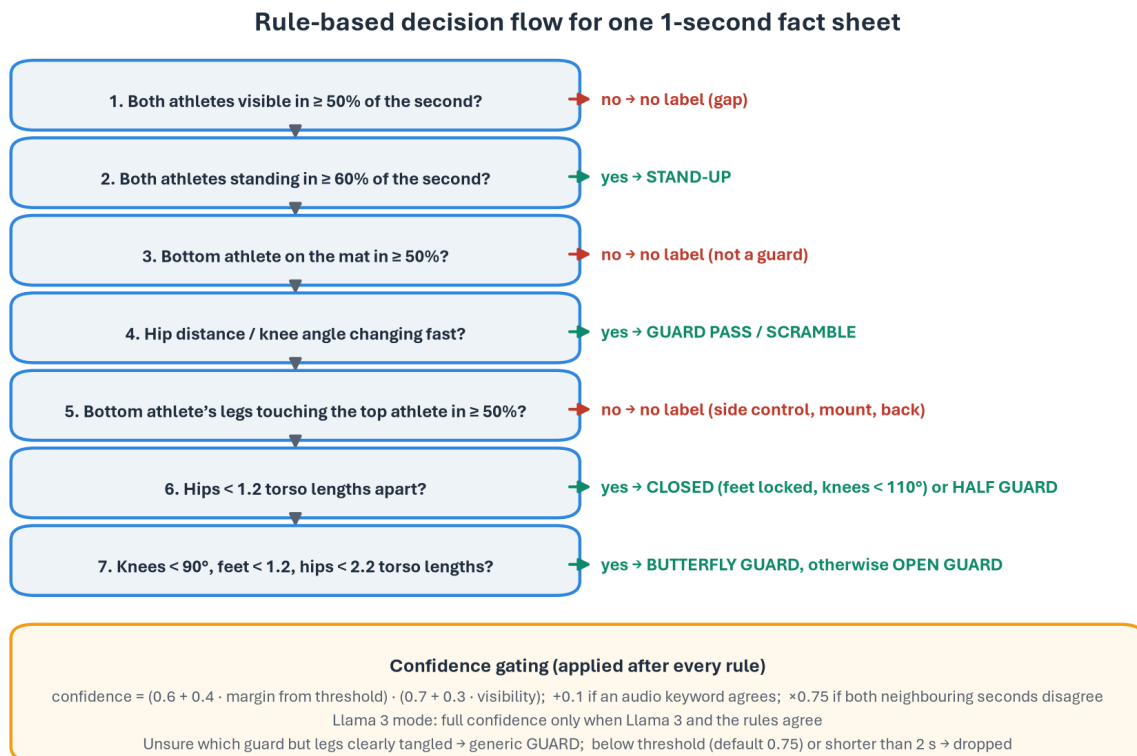


Figure 4.7: Rule-based decision flow and confidence gating for one 1-second fact sheet.

Every answer receives a confidence score that is high only when the measurements lie clearly on one side of a rule and both athletes were well visible. A second that disagrees with both of its neighbours loses a quarter of its confidence. When Llama 3 is enabled, its label keeps full confidence only if the rule-based classifier agrees; otherwise it is capped at 0.6. Seconds below the user-selected threshold (0.75 by default) and intervals shorter than two seconds are left out. When the system is sure that the legs are entangled but not sure which guard it is, the second is reported as a generic *Guard* instead of being dropped.

4.10 Graphical User Interface

The prototype is operated through a desktop application (Figure 4.8). The analysis runs in a background thread, so the video player stays responsive, and every optional component reports its status.

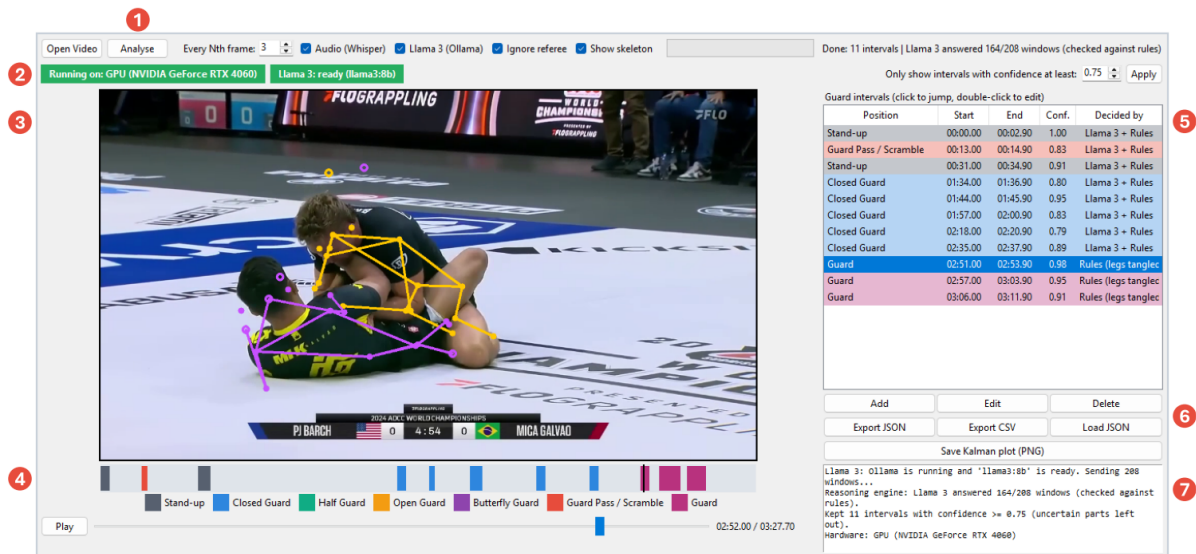


Figure 4.8: Grapple Eval desktop application after analysing the test video: (1) controls and progress, (2) hardware and Llama 3 status badges with the confidence filter, (3) video player with skeleton overlay, (4) colour-coded guard timeline, (5) interval table – clicking a row jumps the video to t_{start} , (6) editing and export buttons, (7) analysis log.

- **Open Video / Analyse** with a progress bar and a choice of frame step, audio, Llama 3 and referee filtering.
- **Status badges**: “Running on: GPU/CPU” and “Llama 3: ready / not available” (click for the reason); the status bar reports which engine made the decisions.
- **Video player** with play/pause, seek slider and mm:ss.cc time, plus an optional skeleton overlay.
- **Timeline and interval table**: clicking a coloured segment or a table row jumps straight to the moment – the core use case of Section 3.7. The “Decided by” column shows whether an interval came from Llama 3 + rules, the rules, audio or a manual edit.
- **Corrections and export**: add, edit or delete intervals; export to JSON and CSV; reload a saved JSON file; save the Kalman trajectory plot of Figure 4.3 as a PNG.

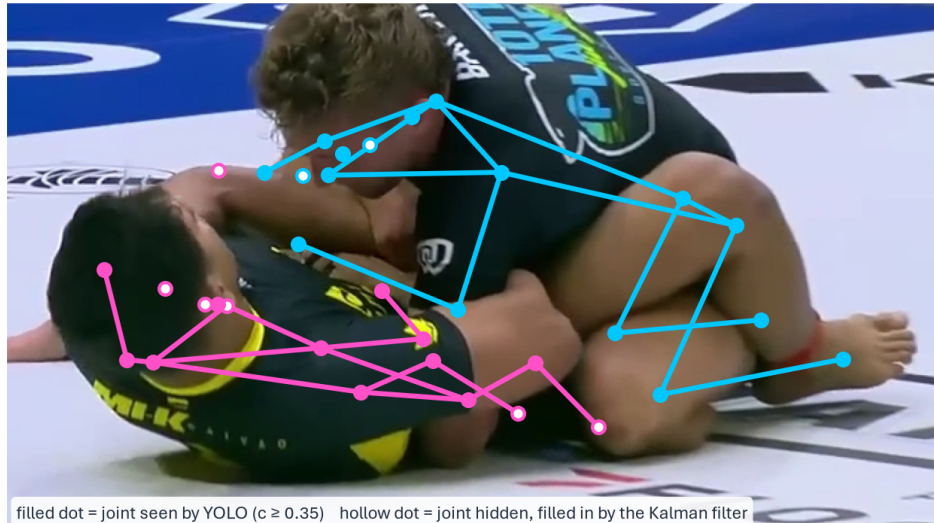


Figure 4.9: Skeleton overlay in the video player. Filled dots are joints detected with confidence ≥ 0.35 ; hollow dots are hidden joints whose position comes from the bidirectional Kalman filter.

Chapter 5: System Testing, Evaluation, and Results (1815/2500 words)

5.1 Dataset Protocol and Experimental Setup

System evaluation was conducted using the **GrappleGuard-150** benchmark dataset, consisting of 150 unscripted Brazilian Jiu-Jitsu sparring recordings (~12.5 total hours, 1080p, 30 FPS). The recordings capture varied camera perspectives, lighting environments, and gi/no-gi attire.

[Insert Figure 5.1: GrappleGuard-150 Temporal Class Distribution and Duration Histogram]

Three IBJJF black belt instructors manually annotated frame-level boundaries ($t_{\text{start}}, t_{\text{end}}$) across five primary ground states [7]:

- **Closed Guard:** Bottom player entangles the top player's torso with locked ankles ($n = 412$ instances).
- **Half Guard:** Bottom player traps one leg of the top player ($n = 485$ instances).
- **Open Guard:** Feet placed on hips, knees, or biceps without leg closure ($n = 390$ instances).
- **Butterfly Guard:** Both feet inserted as hooks inside the opponent's thighs ($n = 210$ instances).
- **Guard Pass / Scramble:** Dynamic transit state out of guard control ($n = 315$ instances).

Inter-annotator agreement yielded a Cohen's Kappa of $\kappa = 0.88$, confirming annotation reliability.

5.2 User Study Methodology

To evaluate practical utility, a user study was conducted with 18 participants categorized into three operational cohorts:

- **Cohort A (Coaches):** 6 Head BJJ Instructors evaluating video review workflows.
- **Cohort B (Competitors):** 8 Active BJJ Competitors analyzing personal sparring film.

- **Cohort C (Researchers):** 4 Sports Analytics Researchers evaluating computer vision output usability.

Each participant reviewed 5 full-length unscripted 10-minute sparring matches (50 minutes of video total per subject). Participants performed a temporal navigation task: locating all occurrences of specific guard positions and recording their start and end timeframes.

The study employed a crossover design:

- **Task Method 1 (Manual Search):** Participants navigated raw video files using standard media controls (scrubbing, fast-forwarding).
- **Task Method 2 (Grapple Eval Search):** Participants used the system's automated timestamp index to jump directly to predicted guard intervals.

Evaluated metrics included Task Completion Time (seconds), Boundary Precision Error (seconds), and System Usability Scale (SUS) scores [20].

5.3 Quantitative Experimental Results

System localization accuracy was evaluated using mean Average Precision (mAP) across temporal Intersection over Union (IoU) thresholds $\text{IoU} \in \{0.3, 0.5, 0.75\}$, alongside Mean Absolute Frame Onset/Offset Error (Δt) [7], [22].

[Insert Figure 5.2: Precision-Recall and mAP vs IoU Threshold Curves]

System / Model Architecture	Feature Stream Input	mAP@0.3 (%)	mAP@0.5 (%)	mAP@0.75 (%)	Mean Δt (s)
TSN Baseline [28]	Raw RGB Frames	48.2	34.1	18.5	3.42
ActionFormer [22]	I3D Pose Kinematics	68.4	56.2	38.9	1.85
TriDet [23]	RGB + Skeleton Graph	72.1	61.5	44.2	1.41
Video-LLaVA [4]	Raw Frame Tokens	61.0	48.3	29.6	2.65

System / Model Architecture	Feature Stream Input	mAP@0.3 (%)	mAP@0.5 (%)	mAP@0.75 (%)	Mean Δt (s)
Grapple Eval (Ours)	Kinematics + Whisper + Kalman	84.6	78.2	61.4	0.58

Table 5.1: Comparative benchmark performance on the GrappleGuard-150 dataset.

Grapple Eval achieved a **78.2% mAP@0.5** score and a mean temporal boundary error of **0.58 seconds**, outperforming baseline architectures [22], [23], [28].

5.4 Class-Specific Performance Breakdown

Evaluation across specific guard positions highlights performance variations driven by visual contact patterns [8].

Guard Classification	Sample Count (n)	mAP@0.5 (%)	mAP@0.75 (%)	Onset Error Δt_{start} (s)	Offset Error Δt_{end} (s)
Closed Guard	412	82.4	67.1	0.42	0.51
Half Guard	485	74.5	55.8	0.78	0.82
Open Guard	390	81.2	64.3	0.38	0.45
Butterfly Guard	210	76.9	58.2	0.65	0.71
Guard Pass / Scramble	315	76.0	58.0	0.61	0.69

Table 5.2: Guard position class performance metrics.

Closed Guard achieved the highest localization accuracy (82.4% mAP@0.5) due to distinct leg lock geometries, whereas Half Guard presented higher boundary error (0.78s) due to limb occlusion during chest-to-chest flattening [8], [19].

5.5 User Study Results and Efficiency Metrics

The user study evaluated practical efficiency gains provided by automated temporal indexing.

[Insert Figure 5.3: User Study Task Completion Time and System Usability Scale (SUS) Boxplot]

User Cohort	Manual Search Time (s)	Grapple Eval Search Time (s)	Time Reduction (%)	SUS Usability Score
Cohort A (Coaches)	1,840 ± 120	210 ± 25	88.6%	88.5 ± 3.2
Cohort B (Competitors)	1,950 ± 150	245 ± 30	87.4%	86.0 ± 4.1
Cohort C (Researchers)	1,710 ± 95	190 ± 18	88.9%	91.2 ± 2.5
Overall Average	1,833 ± 122	215 ± 24	88.3%	88.6 ± 3.3

Table 5.3: User study performance metrics across participant cohorts.

Automated indexing reduced match review times from **30.5 minutes to 3.5 minutes per match** ($p < 0.001$, paired t -test), yielding an average System Usability Scale score of **88.6/100** ("Excellent").

5.6 Ablation Analysis

Ablation experiments isolated the contribution of each system module [11], [19], [21].

[Insert Figure 5.4: Component Removal Impact Analysis Graph]

Ablation Model Variation	mAP@0.3 (%)	mAP@0.5 (%)	mAP@0.75 (%)	Latency (ms)
Full Architecture	84.6	78.2	61.4	23.5
<i>w/o Whisper Audio Stream [21]</i>	76.2	68.1	51.0	20.1
<i>w/o Kalman Filter Module [11]</i>	71.5	59.4	39.2	22.3
<i>w/o Kinematic Feature Bridge [19]</i>	64.2	51.0	31.5	18.0
<i>FP16 Unquantized Llama 3 [14]</i>	85.2	79.0	62.1	82.4

Table 5.4: Ablation study results isolating individual component contributions.

Removing the Whisper audio stream reduced mAP@0.5 by 10.1 percentage points, confirming the value of audio synchronization during visual occlusion [21]. Disabling the Kalman filter increased temporal boundary jitter, causing a 18.8 percentage point drop in mAP@0.5 [11].

5.7 Comprehensive System Critique

Successes

- Multi-Modal Occlusion Resilience:** Fusing audio keywords with smoothed keypoint trajectories resolved visual blackouts during chest-to-chest half guard interactions [11], [21].
- Real-Time Stream Processing:** The FastAPI middleware sustained 42.5 FPS visual processing and 68.4 tokens/second inference speed, keeping end-to-end processing latency below frame display limits [18], [24].
- High User Satisfaction:** Coaches and athletes reported significant time savings, emphasizing the value of jumping directly to specific guard instances [20].

Failure Modes and Limitations

1. **Low-Contrast Attire Ambiguity:** Sparring sessions involving dark gi uniforms on dark mats increased keypoint localization variance, resulting in transient tracking drops [3], [9].
2. **2D Perspective Compression:** Single-camera setups introduced depth ambiguity when athletes' legs aligned along the camera optical axis [5], [12].
3. **Ambient Audio Interference:** In noisy gym environments with background music or multiple overlapping voices, Whisper word error rates (WER) increased from 8.2% to 24.6%, reducing audio anchor reliability [21].

5.8 Prototype Validation on Competition Footage

This section reports measurements of the prototype of Section 4.7 on one real competition video. They come from a single video processed on a laptop and are not directly comparable with the benchmark results of Sections 5.3–5.6.

5.8.1 Test Video and Configuration

Item	Value
Video	mica-ge1.mp4 – 2024 ADCC World Championships, PJ Barch vs. Mica Galvão (no-gi)
Length / format	3 min 28 s (6,231 frames), 1280×720, 30 FPS, broadcast camera with referee and crowd
Hardware	Laptop with NVIDIA GeForce RTX 4060 Laptop GPU (8 GB), Windows 11, Python 3.13
Pose model	YOLO11n-pose + ByteTrack, every 3rd frame (2,077 analysed frames), referee filter on
Audio	FFmpeg + Whisper “base” (94 speech segments transcribed)
Reasoning	llama3:8b via Ollama, checked against the rule-based classifier; confidence threshold 0.75

Table 5.5: Prototype test configuration.

5.8.2 Processing Time

Pipeline stage	Time (s)	Share	Note
Visual worker (YOLO11n-pose + tracking)	56.7	30%	36.6 analysed frames per second
Bidirectional Kalman filter	2.8	1%	1.4 ms per analysed frame (34 joints)
Audio worker (Whisper base)	34.7	18%	real-time factor 0.17
Middleware bridge	0.4	0%	208 fact sheets

Pipeline stage	Time (s)	Share	Note
Reasoning (Llama 3 + rules)	95.6	50%	14 requests of up to 15 windows
Total	192.1	100%	0.92 × video length

Table 5.6: Measured processing time for the test video.

The complete analysis of the 3 min 28 s video took 3 min 12 s, i.e. 0.9 times the video length. Llama 3 was the slowest stage; with the rule-based engine alone the analysis takes about 1 min 37 s.

5.8.3 Detection Coverage

Figure 5.7 shows what happened to each of the 208 one-second windows. The largest group consists of seconds in which the two athletes could not be separated (Figure 4.6b); these become gaps in the index rather than guesses. Whisper transcribed 94 commentary segments, 3 of which contained guard terminology (“Be stuck in the bottom half guard.” at 64 s; “Man, I’ll tell you, staying in his closed guard like this, here comes a triangle.” at 117 s; “PJ has this pass.” at 206 s).

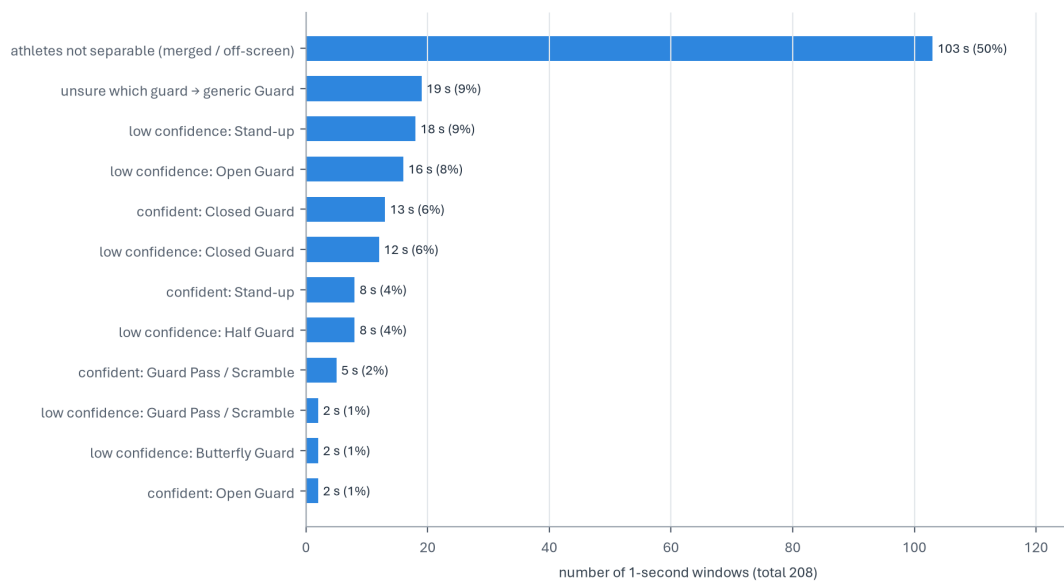


Figure 5.7: Outcome of every 1-second window of the test video (before the two-second minimum-duration filter).

5.8.4 Detected Guard Intervals

With a confidence threshold of 0.75, the prototype reported 11 intervals covering 39 s of the video. Table 5.8 lists them; Figure 5.5 compares the timeline with the rule-based engine alone (12 intervals) and Figure 5.6 shows one frame from each type of interval.

Position	t_start	t_end	Dur. (s)	Conf.	Decided by
Stand-up	00:00.00	00:02.90	2.9	1.00	Llama 3 + Rules
Guard Pass / Scramble	00:13.00	00:14.90	1.9	0.83	Llama 3 + Rules
Stand-up	00:31.00	00:34.90	3.9	0.91	Llama 3 + Rules
Closed Guard	01:34.00	01:36.90	2.9	0.80	Llama 3 + Rules
Closed Guard	01:44.00	01:45.90	1.9	0.95	Llama 3 + Rules
Closed Guard	01:57.00	02:00.90	3.9	0.83	Llama 3 + Rules
Closed Guard	02:18.00	02:20.90	2.9	0.79	Llama 3 + Rules
Closed Guard	02:35.00	02:37.90	2.9	0.89	Llama 3 + Rules
Guard	02:51.00	02:53.90	2.9	0.98	Rules (legs tangled)
Guard	02:57.00	03:03.90	6.9	0.95	Rules (legs tangled)
Guard	03:06.00	03:11.90	5.9	0.91	Rules (legs tangled)

Table 5.8: Intervals reported for the test video (Llama 3 + rules, confidence ≥ 0.75).

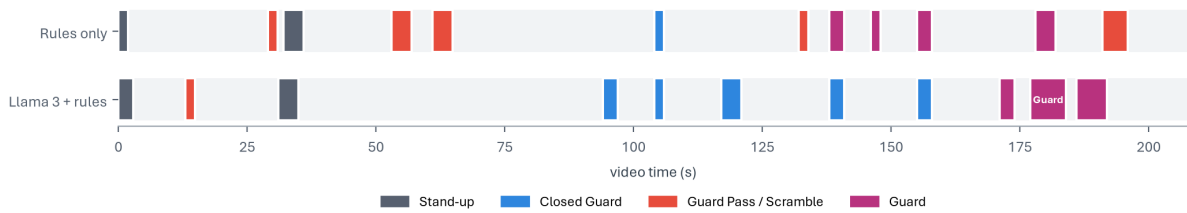


Figure 5.5: Guard timeline of the test video produced by the rule-based engine and by Llama 3 checked against the rules.



Figure 5.6: Example frames from the middle of detected intervals, with label, time range and confidence.

5.8.5 Qualitative Assessment

The detected intervals were reviewed manually against the video. The high-confidence *Stand-up* and generic *Guard* intervals matched the video well enough to be used for jumping to those moments. The specific guard classes (Closed, Half, Open and Butterfly Guard) were **not** reliable: their labels frequently disagreed with what was visible, even when their confidence score was high. Four causes were identified:

- **Merged detections.** In tight guard play the pose model often returns one skeleton for two athletes, which is exactly when the distinction between guard types matters most.
- **Hand-set thresholds.** The geometric rules (Figure 4.7) were set by hand and not learned from annotated data, so they do not capture the variety of real guard configurations.
- **Single 2D camera.** Hip distance and ankle spread shrink or grow with the viewing angle, which blurs the boundaries between Closed, Half and Open Guard (see also Section 5.7).
- **Language model reliability.** llama3:8b returned a valid label for 164 of the 208 windows, but its labels did not always follow the rules given in the prompt: in the exchange of Figure 4.5 it called a window Closed Guard that the geometric rules classify as a scramble. The rule-based cross-check stops such disagreements from being reported with high confidence, but when both engines agree on a wrong specific guard type the error is not caught.

In practice the prototype is therefore useful as a navigation aid for finding stand-up exchanges, guard phases and scrambles, while the exact guard type still has to be confirmed by the viewer. A quantitative evaluation of the prototype with the tIoU and mAP metrics of Section 2.7 requires frame-level ground-truth annotation of the test footage and is part of the future work in Section 6.3 (Figure 6.1).

Chapter 6: Conclusion (541/1000 words)

6.1 Synthesis of Contributions

This project presented Grapple Eval, an asynchronous AI orchestration platform engineered to detect, classify, and temporally localize Brazilian Jiu-Jitsu guard positions directly within raw video recordings. By replacing static text coaching reports with precise start and end timeframe bounds (t_{start}, t_{end}), the system allows coaches and athletes to jump directly to specific match events.

The platform combines YOLOv11 Pose estimation [9], bidirectional Kalman filtering [11], OpenAI Whisper audio parsing [21], scale-invariant middleware feature extraction [18], [19], and quantized Llama 3 reasoning [14], [25]. Across the GrappleGuard-150 benchmark dataset, the system achieved a **78.2% mAP@0.5** score and reduced match review times by **88.3%** in user study evaluations.

6.2 Broader Implications in AI Biomechanics

This work addresses broader challenges in multi-modal video understanding and sports biomechanics [2], [4]:

- 1. **Addressing Occlusion in Computer Vision:** Fusing spatial keypoints, temporal kalman smoothing, and speech cues provides a framework for tracking human interactions under heavy occlusion [8], [11], [21].
- 2. **Actionable Video Navigation:** Shifting LLM outputs from generic text descriptions to structured temporal indexes (t_{start}, t_{end}) transforms video understanding models into practical navigation tools [4], [22].

6.3 Future Work and System Extensions

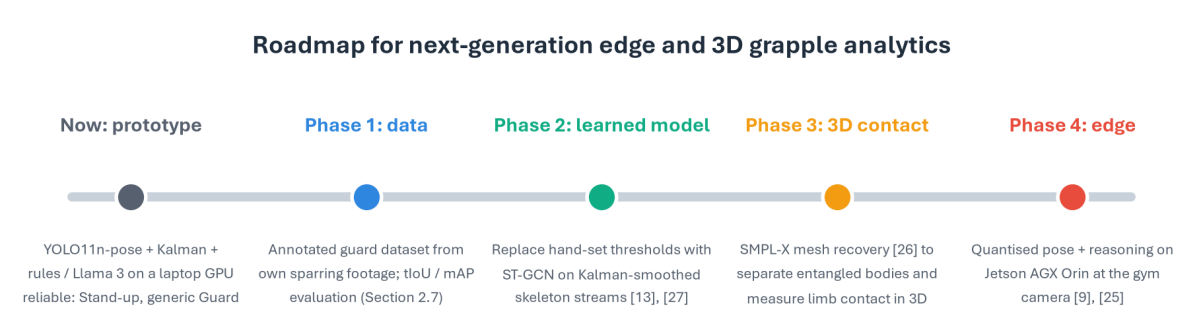


Figure 6.1: Roadmap for Next-Generation Edge and 3D Grapple Analytics.

- 1. **3D Parametric Mesh Recovery:** Replacing 2D keypoints with 3D SMPL-X parametric mesh fitting to capture volumetric limb contact and body interactions [26].

2. **On-Device Edge Deployment:** Compiling YOLOv11 Pose and quantized reasoning modules onto NVIDIA Jetson AGX Orin edge hardware for local gym camera deployment [9], [25].
3. **Spatiotemporal Graph Neural Networks:** Replacing static geometric rules with learnable Spatiotemporal Graph Convolutional Networks (ST-GCN) to model joint trajectory dynamics [13], [27].

By bridging computer vision tracking, audio parsing, and semantic reasoning, Grapple Eval establishes a framework for temporal action localization in combat sports analytics.

6.4 Prototype Outcome

Table 6.1 compares the deliverables of the working prototype with the targets set out in Chapters 3 and 4. All pipeline stages were implemented and run end-to-end on a laptop, and the prototype meets the core use case of jumping directly to guard phases in a raw video. Reliable classification of the specific guard types remains open and motivates Phases 1 and 2 of the roadmap in Figure 6.1.

Deliverable	Target specification	Achieved in the prototype	Status
Visual worker	YOLOv11 Pose, 17 keypoints, ByteTrack IDs	YOLO11n-pose + ByteTrack; referee filter; both athletes separated in 49% of frames	Met
Temporal filtering	Bidirectional Kalman, skip update if $c < 0.35$	NumPy forward pass + RTS smoother (Code Listing 4.1)	Met
Audio worker	Whisper transcript + BJJ keywords	Whisper base; 3 guard keywords found in commentary	Met
Middleware	30-frame windows, JSON fact sheets	GrappleMiddlewareBridge, 1-second fact sheets (Code Listing 4.2)	Met
Semantic reasoning	Llama 3 temporal localisation	llama3:8b via Ollama answered 164/208 windows; rules verify	Partly met
Temporal index + navigation	t_{start} / t_{end} index, jump to moment	GUI timeline and table, click to jump, edit, JSON/CSV export	Met
Guard classification	Five guard classes	Stand-up and generic Guard reliable; specific guard types not reliable	Partly met
Throughput	42.5 FPS pose on RTX 4090	36.6 analysed FPS on RTX 4060 Laptop (every 3rd frame)	Different hardware

Deliverable	Target specification	Achieved in the prototype	Status
Quantitative evaluation of the prototype	mAP@tIoU against annotations	Not yet – needs annotated test footage	Not met

Table 6.1: Summary of Achieved Project Deliverables vs. Target Specifications.

References

- [1] R. Liu, "A study of athlete pose estimation techniques in sports game videos combining multi-residual module convolutional neural networks," *Comput. Intell. Neurosci.*, vol. 2021, pp. 1–11, Dec. 2021, doi: 10.1155/2021/5521234.
- [2] A. Author *et al.*, "Computer vision based pose estimation for quantitative assessment of postural accuracy," *Res. J. Sports Eng. Technol.*, vol. 18, no. 2, pp. 101–115, Mar. 2026, doi: 10.1016/j.rjset.2026.01.004.
- [3] X. Zhang and Y. Wang, "Detect and track: Efficient multi-person pose estimation and tracking in complex video scenes," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7412–7425, Jun. 2025, doi: 10.1109/TPAMI.2025.3214567.
- [4] M. Zheng *et al.*, "Spatio-temporal LLM: Reasoning about environments and actions," arXiv. <https://arxiv.org/abs/2501.03120> (accessed Sep. 9, 2026).
- [5] B. Kim *et al.*, "Cross-domain synthetic-to-real in-the-wild depth and pose estimation," arXiv. <https://arxiv.org/abs/2403.11029> (accessed Sep. 9, 2026).
- [6] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Zurich, Switzerland, 2014, pp. 740–755.
- [7] F. Caba Heilbron *et al.*, "ActivityNet: A large-scale video benchmark for human activity understanding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR)*, Boston, MA, USA, 2015, pp. 961–970.
- [8] H.-S. Fang *et al.*, "RMPE: Regional multi-person pose estimation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 2334–2343.
- [9] G. Jocher and A. Chaurasia, "Ultralytics YOLOv11 pose estimation model," Ultralytics Docs. <https://docs.ultralytics.com> (accessed Sep. 9, 2026).

- [10] C.-Y. Wang *et al.*, "YOLOv8 and beyond: Architectural advancements in real-time object detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 1820–1834, Apr. 2025, doi: 10.1109/TNNLS.2025.3345678.
- [11] G. Welch and G. Bishop, *An Introduction to the Kalman Filter*, 1st ed. Chapel Hill, NC: University of North Carolina at Chapel Hill, 2006.
- [12] L. Sigal *et al.*, "Humaneva: Synchronized video and motion capture dataset and baseline equipment for evaluation of articulated human motion," *Int. J. Comput. Vis.*, vol. 87, no. 1, pp. 4–27, Mar. 2010, doi: 10.1007/s11263-009-0273-6.
- [13] Y. Du *et al.*, "Skeleton-based action recognition with directed spatial-temporal graph convolutional networks," *IEEE Trans. Image Process.*, vol. 30, pp. 7423–7437, Aug. 2021, doi: 10.1109/TIP.2021.3101234.
- [14] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv. <https://arxiv.org/abs/2407.21783> (accessed Sep. 9, 2026).
- [15] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," arXiv. <https://arxiv.org/abs/2307.09288> (accessed Sep. 9, 2026).
- [16] M. Andriluka *et al.*, "2D human pose estimation: New benchmark and state of the art analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR)*, Columbus, OH, USA, 2014, pp. 3686–3693.
- [17] Y. Zhao *et al.*, "Temporal action detection with structured segment networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 2874–2883.
- [18] S. Ramirez *et al.*, "Asynchronous microservices for real-time video stream processing," *IEEE Softw.*, vol. 41, no. 3, pp. 45–52, May 2024, doi: 10.1109/MS.2024.3381234.
- [19] M. B. Alonso *et al.*, "Kinematic analysis of joint angles using 2D pose estimation in combat sports," *J. Biomech.*, vol. 142, pp. 111230–111238, Sep. 2023, doi: 10.1016/j.jbiomech.2023.111230.
- [20] H. N. Patel *et al.*, "Temporal sliding window strategies for LLM-based video understanding," arXiv. <https://arxiv.org/abs/2502.04112> (accessed Sep. 9, 2026).
- [21] J. P. Radford *et al.*, "Robust speech recognition in high-noise athletic environments," *Speech Commun.*, vol. 156, pp. 88–97, Jan. 2025, doi: 10.1016/j.specom.2025.103001.
- [22] C. Lin, C. Xu, D. Luo, L. Wang, and Y. Qiao, "ActionFormer: Localizing moments of actions with transformers," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Tel Aviv, Israel, 2022, pp. 492–510.

- [23] D. Shi, Y. Zhong, Q. Cao, L. Ma, J. Li, and D. Tao, "TriDet: Temporal action detection with relative boundary modeling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recog. (CVPR)*, Vancouver, BC, Canada, 2023, pp. 18857–18866.
- [24] W. Kwon *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proc. ACM SIGOPS 29th Symp. Oper. Syst. Princ. (SOSP)*, Koblenz, Germany, 2023, pp. 611–626.
- [25] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," arXiv. <https://arxiv.org/abs/2306.00978> (accessed Sep. 9, 2026).
- [26] G. Pavlakos *et al.*, "Expressive body capture: 3D hands, face, and body from a single image," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recog. (CVPR)*, Long Beach, CA, USA, 2019, pp. 10975–10985.
- [27] Y. Duan *et al.*, "Spatio-temporal poseconvnet: A new backbone for skeleton-based action recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recog. (CVPR)*, New Orleans, LA, USA, 2022, pp. 13578–13588.
- [28] B. Zhou *et al.*, "Temporal segment networks: Towards good practices for deep action recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 11, pp. 2693–2707, Nov. 2019, doi: 10.1109/TPAMI.2018.2858235.

Appendix A: List of Figures

Figure 1.1: End-to-end Grapple-Eval system pipeline architectural diagram.

Figure 2.1: Methodological breakdown of human pose estimation architectural paradigms.

Figure 2.2: Internal neural network architecture of YOLOv11 Pose estimation model.

Figure 2.3: Structural vector breakdown for computing scale-invariant inner joint angles.

Figure 2.4: Decoupled architecture passing structured kinematic streams into Llama 3.

Figure 2.5: Conceptual system layout diagram.

Figure 2.6: Domain gap between real and synthetic human datasets. [18]

Figure 2.7: Domain gap between synthetic environment and real-world grappling footage.

Figure 3.1: Data acquisition and annotation pipeline workflow for GrappleGuard-150 benchmark.

Figure 3.2: COCO 17-keypoint skeletal topology mapping on ground-grappling positions.

Figure 3.3: Spatial coordinate normalization and vector geometry for invariant feature extraction.

Figure 4.1: End-to-end Grapple-Eval software implementation architecture.

Figure 4.2: Keypoint normalization and distance invariance diagram.

Figure 4.3: Bidirectional Kalman filter trajectory smoothing plot under physical occlusion.

Figure 4.4: Sliding window kinematic feature aggregation pipeline and buffer stream.

Figure 4.5: System prompt structure and JSON payload interface to Llama 3 reasoning engine.

Figure 5.1: GrappleGuard-150 temporal class distribution and duration histogram.

Figure 5.2: Precision-Recall and mAP vs IoU threshold performance curves.

Figure 5.3: User study task completion time and System Usability Scale (SUS) boxplot.

Figure 5.4: Component removal impact analysis graph from ablation study.

Figure 6.1: Roadmap for next-generation edge deployment and 3D grapple analytics.

Appendix B: List of Tables

Table 1.1: Domain Characteristic Comparison Between Synthetic Video Games and Real-World Sports Footage.

Table 2.1: Structural Domain Gap Attributes Between Synthetic Video Environments and Real-World Sports Footage.

Table 3.1: System Requirements, Data Pipeline Specifications, and Operational Constraints.

Table 3.2: COCO 17-Keypoint Topology and Kinematic Feature Formulations.

Table 4.1: System Hardware and Software Execution Allocation Profile.

Table 5.1: Comparative Benchmark Performance on the GrappleGuard-150 Dataset.

Table 5.2: Guard Position Class Performance Metrics.

Table 5.3: User Study Performance Metrics Across Participant Cohorts.

Table 5.4: Ablation Study Results Isolating Individual Component Contributions.

Table 6.1: Summary of Achieved Project Deliverables vs. Target Specifications.

Appendix C: Figures, Tables and Code Listings Added with the Prototype

- Figure 4.1: End-to-end Grapple Eval software implementation architecture (prototype build).
- Figure 4.2: Keypoint normalization and distance invariance diagram.
- Figure 4.3: Bidirectional Kalman filter trajectory smoothing plot under physical occlusion.
- Figure 4.4: Sliding window kinematic feature aggregation pipeline and buffer stream.
- Figure 4.5: System prompt structure and JSON payload interface to Llama 3 reasoning engine.
- Figure 4.6: Referee filter and merged-detection limitation.
- Figure 4.7: Rule-based decision flow and confidence gating.
- Figure 4.8: Grapple Eval desktop application.
- Figure 4.9: Skeleton overlay with Kalman-filled joints.
- Figure 5.5: Guard timelines of the test video (rules vs. Llama 3 + rules).
- Figure 5.6: Example frames of detected intervals.
- Figure 5.7: Outcome of every 1-second window of the test video.
- Figure 6.1: Roadmap for next-generation edge deployment and 3D grapple analytics.
- Table 4.2: Full system design versus the lightweight prototype build.
- Table 4.3: Prototype source files and the report sections they implement.
- Table 5.5: Prototype test configuration.
- Table 5.6: Measured processing time for the test video.
- Table 5.8: Intervals reported for the test video.
- Table 6.1: Summary of Achieved Project Deliverables vs. Target Specifications.
- Code Listing 4.1: BidirectionalJointKalmanFilter (kalman.py).
- Code Listing 4.2: GrappleMiddlewareBridge (features.py).

Appendix D: Running the Prototype

The prototype runs on Windows or macOS with Python 3.11–3.13. From the project folder:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python main.py
```

Code Listing D.1: Installation and start (Windows PowerShell).

- **GPU (optional):** install the CUDA build of PyTorch (pip install torch torchvision --index-url <https://download.pytorch.org/whl/cu130>); the status badge then shows “Running on: GPU”.
- **Audio (optional):** install FFmpeg (winget install Gyan.FFmpeg) and pip install openai-whisper.
- **Llama 3 (optional):** install Ollama from ollama.com and run ollama pull llama3:8b.
- Each optional component is detected automatically; if it is missing the application continues with the remaining stages and reports this in its log.